Università degli Studi di Modena e Reggio Emilia

FACOLTÀ DI INGEGNERIA
Corso di Laurea in Ingegneria Informatica

Prova Finale

Studio di un Giocatore Artificiale del Gioco degli Scacchi

Candidato:
Francesco Martucci
Matricola 119082

Relatore:
Nicola Bicocchi

Anno Accademico 2019-2020

All'unione di Shiva e Parvati che rende feconda la distruzione.

Indice

1 Introduzione **7**

1.1 Panoramica 7

1.2 Perché gli scacchi? 8

1.3 Cosa sono gli Scacchi? 9

1.3.1 La Teoria dei Giochi 9

1.3.2 Il Gioco degli Scacchi 10

1.3.3 L'albero di gioco 11

1.4 Cos'è un motore scacchistico? 14

1.4.1 Descrizione funzionale 14

1.4.2 Descrizione strutturale 15

1.5 Stato dell'arte 17

2 Rappresentazione e Generazione **20**

2.1 Cosa rappresentare? 20

2.2 Rappresentazione della scacchiera e dei pezzi 21

2.2.1 Rappresentazione orientata alla scacchiera 21

2.2.2 Rappresentazione orientata ai pezzi 24

2.2.3 Bitboards 25

2.3 Generazione delle mosse 27

2.3.1 Rappresentazione delle mosse 27

2.3.2 Generazione delle mosse 28
2.3.3 Vettori di Movimento 29
2.3.4 Mappe dei Movimenti 29

3 Ricerca **32**
3.1 Qual è lo scopo della ricerca? 32
3.2 Make-Undo Move 33
3.3 Minimax e Alphabeta 34
3.3.1 Minimax 34
3.3.2 Effetto orizzonte 36
3.3.3 Alphabeta 37
3.3.4 Efficienza di AlphaBeta 40
3.3.5 Ordinamento delle mosse 41
3.3.6 Aspiration Search 42
3.3.7 History Heuristic 43
3.3.8 Null Move Heuristic 44
3.3.9 Selective Extension e Selective Reduction 44
3.3.10 Tabella delle Trasposizioni 45
3.3.11 Hashing di Zobrist 47
3.3.12 Lazy Evaluation 50
3.4 Approcci alternativi 50
3.4.1 Iterative Deepening 50
3.4.2 Numeri di Cospirazione 52
3.4.3 Monte Carlo 53
3.4.4 Parallelizzazione 54

4 Valutazione **56**
4.1 Qual è lo scopo della valutazione? 56

4.2 La Funzione di Valutazione 57
4.2.1 Materiale 57
4.2.2 Posizione 58
4.3 Considerazioni 60

5 Progetto personale 62
5.1 Qual è lo scopo del mio progetto personale? 62
5.2 Requisiti da soddisfare 62
5.3 Panoramica sull'architettura 63
5.4 Rappresentazione e generazione 64
5.4.1 Rappresentazione a oggetti e generatore legale 65
5.4.2 Rappresentazione ibrida e generatore pseudo-legale . . 65
5.5 Ricerca 67
5.6 Valutazione 68
5.7 Debugging e testing 69
5.8 Risultati 70
5.8.1 Confronto fra i generatori 71
5.8.2 Confronto fra gli algoritmi 72
5.8.3 Confronto fra le funzioni di valutazione 73

Elenco delle figure

1.1 Come appare l'albero di gioco degli scacchi [18] 12
1.2 Activity diagram di un motore scacchistico 16
1.3 Communication diagram di un motore scacchistico 16
1.4 Component diagram di un motore scacchistico 18

2.1 Per ogni casa, è indicato l'indice che la identifica. Fonte: ChessProgramming Wiki 23
2.2 Rappresentazione 12x10 nel motore COKO III 24
2.3 Rappresentazione della scacchiera secondo 0x88. Gli indici invalidi sono quelli presenti nelle celle più scure. Gli indici sono presentati in notazione esadecimale 25
2.4 Esempio di bitboard dei pedoni bianchi 26
2.5 Esempio di bitboard dei pedoni bianchi sulla scacchiera 26
2.6 Bitboard di movimento della torre 30
2.7 Bitboard di cattura del cavallo 30

3.1 Tipi di nodo Alphabeta 41
3.2 Esempio di trasposizione 45

5.1 Component diagram del motore 63
5.2 Class Diagram dell'Esploratore 68

Elenco delle tabelle

Rappresentazione orientata alla scacchiera 22

Valore dei pezzi 58

Risultati Perft 70

Confronto tra generatori 71

Percentuali dei tempi delle fasi di ricerca 71

Percentuale nodi CUT 72

Miglioramento con T.T. e percentuale hit 72

Capitolo 1

Introduzione

1.1 Panoramica

Questo elaborato di tesi sintetizza il lavoro da me svolto durante l'attività progettuale interna. Qui, presenterò le più importanti idee che hanno caratterizzato la programmazione scacchistica dalla sua nascita ad oggi, cercando di lasciare al lettore una panoramica sul lavoro svolto in tale ambito. L'argomento è vasto e complesso, per cui, in questa sede, sarà possibile solo scalfirne la superficie.

Nel seguito di questo capitolo, si esporranno gli scopi e le basi teoriche della programmazione scacchistica, arrivando a descrivere cosa siano un giocatore artificiale ed un motore scacchistico; infine, si darà una panoramica dello stato dell'arte attuale. Nei capitoli successivi si scenderà nel dettaglio dell'implementazione di un motore, analizzando ciascun componente, problemi implementativi affrontati, soluzioni individuate, funzionamento e abbozzo della struttura. Nell'ultimo capitolo, presenterò brevemente il progetto personale realizzato: un esempio pratico a tutta la trattazione teorica qui esposta.

1.2 Perché gli scacchi?

L'ambizione di creare una macchina in grado di giocare a scacchi esiste almeno dal XVIII secolo. É possibile citare vari esempi, ma fino all'avvento del moderno calcolatore si riuscì a realizzare giocatori artificiali in grado di giocare solo finali semplici (ad esempio, re contro re e torre). Claude Shannon fu il primo a presentare un progetto chiaro *moderna* di motore scacchistico sfruttando, appunto, il calcolatore. Egli presentò una serie di concetti e considerazioni teoriche le quali sono tutt'oggi applicate [17]. Lo studio di un motore scacchistico appariva interessante: i ricercatori si chiedevano quali erano i componenti fondamentali di un sistema in grado di prendere decisioni, come essi funzionavano e con quali prestazioni. Più in generale, si stava affrontando il problema della rappresentazione della conoscenza. Infatti, per quanto riguarda il gioco degli Scacchi, se ne possono distinguere due tipi:

- dichiarativa: come rappresentare gli elementi e le regole del gioco
- procedurale: quali calcoli intraprendere per il conseguimento di un obiettivo, ad esempio dare scacco matto al re avversario

Altro punto era capire quanta conoscenza del dominio fosse necessario implementare nel sistema stesso (i sistemi detti *agenti esperti* stavano dando buoni risultati, ma si era presto capito che il campo d'applicazione era limitato). In altre parole, si stava cercando di comprendere cosa significasse essere "intelligente". Secondo Shannon, il primo passo era di comprendere come svolgere operazioni tra simboli, invece che tra numeri ed essere in grado di identificare una soluzione sufficientemente buona (non obbligatoriamente la migliore). Il gioco degli Scacchi appariva come un eccellente dominio per il confronto e lo sviluppo di tali studi:

- le regole del gioco sono ben definite

- il dominio può essere facilmente decomposto in sotto-domini per isolare alcuni problemi (ad esempio dedicarsi solo al finale della partita)
- il gioco degli scacchi è molto noto, permettendo di valutare direttamente i risultati del sistema sviluppato affrontando giocatori umani
- il gioco è da sempre considerato di "intelligenza". [1]

1.3 Cosa sono gli Scacchi?

In questa sezione, verrà presentata la descrizione formale del gioco degli Scacchi, secondo la Teoria dei Giochi.

1.3.1 La Teoria dei Giochi

Con Gioco s'intendono le situazioni di conflitto o cooperazione tra due o più partecipanti, i quali interagiscono tra loro e determinano l'esito finale di tale interazione.

In tal senso, la Teoria dei Giochi può essere definita come una disciplina matematica che analizza situazioni di conflitto in cui vi sono interazioni tra più soggetti, tali per cui le decisioni di uno possono influire sui risultati degli altri secondo un meccanismo di retroazione. Ogni decisione è finalizzata al massimo guadagno del soggetto stesso. In altre parole, la Teoria dei Giochi ha l'obiettivo di modellare una situazione di interazione, per permettere di comprendere, simulare e valutare quali saranno i comportamenti e le decisioni (strategie) delle entità coinvolte.

[1] In realtà, affermare che sia un gioco per persone "intelligenti" è piuttosto scorretto: esistono molteplici tipi di intelligenza e gli scacchi riguardano solo alcuni di questi tipi. É ovvio che gli studiosi si pongono in questo modo per provocare e spingere a chiedersi cosa voglia dire intelligenza.

Gli elementi fondamentali per lo studio di un Gioco sono:

- insieme dei partecipanti
- strategie disponibili per ciascuno dei partecipanti
- cosa ottiene ciascun giocatore in ciascuna delle possibili conclusioni del Gioco (payoff) e quali esiti preferisce
- determinare se un giocatore gioca (sceglie) in maniera razionale

Definito un Gioco, la Teoria si propone di rispondere alle seguenti domande:

- quali scelte di strategie faranno i vari giocatori?
- noti gli obiettivi, come si possono ottimizzare le strategie scelte?

1.3.2 Il Gioco degli Scacchi

Nel linguaggio della Teoria dei Giochi, gli Scacchi sono un gioco avente le seguenti caratteristiche:

- due partecipanti in competizione
- deterministico: con un numero finito di mosse e senza elementi casuali
- informazione perfetta: i due avversari conoscono in ogni istante le stesse informazioni sullo stato del gioco
- somma-zero: il vantaggio di un partecipante corrisponde allo svantaggio dell'altro (teorema del minimax)

- mosse sequenziali: i due partecipanti si alternano nell'effettuare una mossa (tale mossa appartiene ad un insieme ben definito e finito di mosse legali)

- finito: la partita termina necessariamente con la vittoria di uno dei due giocatori o in parità, in un numero finito di mosse

1.3.3 L'albero di gioco

Per descrivere graficamente i giochi sequenziali, come gli scacchi, si utilizza un albero di gioco. I nodi sono le posizioni e gli archi le mosse legali. Nel caso degli scacchi, la radice è la posizione iniziale dei pezzi sulla scacchiera; i figli sono le posizioni raggiungibili dalle mosse legali; le foglie sono le posizioni di scaccomatto o parità.

Grazie all'inquadratura formale data dalla Teoria dei Giochi, è possibile dimostrare che esiste un algoritmo in grado di trovare la mossa migliore, data una posizione qualunque. Tale algoritmo non farebbe altro che esplorare l'albero avente come radice la posizione data: considera tutte le mosse a disposizione del colore che deve giocare, poi tutte quelle del colore avversario e così via fino alla fine del gioco, per ogni possibile variazione (cioè sequenza di mosse). Per le regole stesse del gioco, si giungerà ad una foglia in un numero finito di mosse. Tale foglia sarà una vittoria, una sconfitta o un pareggio. Da qui, lavorando a ritroso, è possibile determinare qual è la mossa migliore, cioè quella che porta, appunto, alla vittoria.

Nonostante la semplicità delle regole e il numero limitato di "elementi di gioco" (scacchiera e pezzi), l'albero degli scacchi ha dimensioni esorbitanti. La radice genera venti figli, ma ogni nodo ne genera in media 30-40. Secondo le stime di C.E. Shannon [17], il numero di posizioni differenti è circa 10^{43},

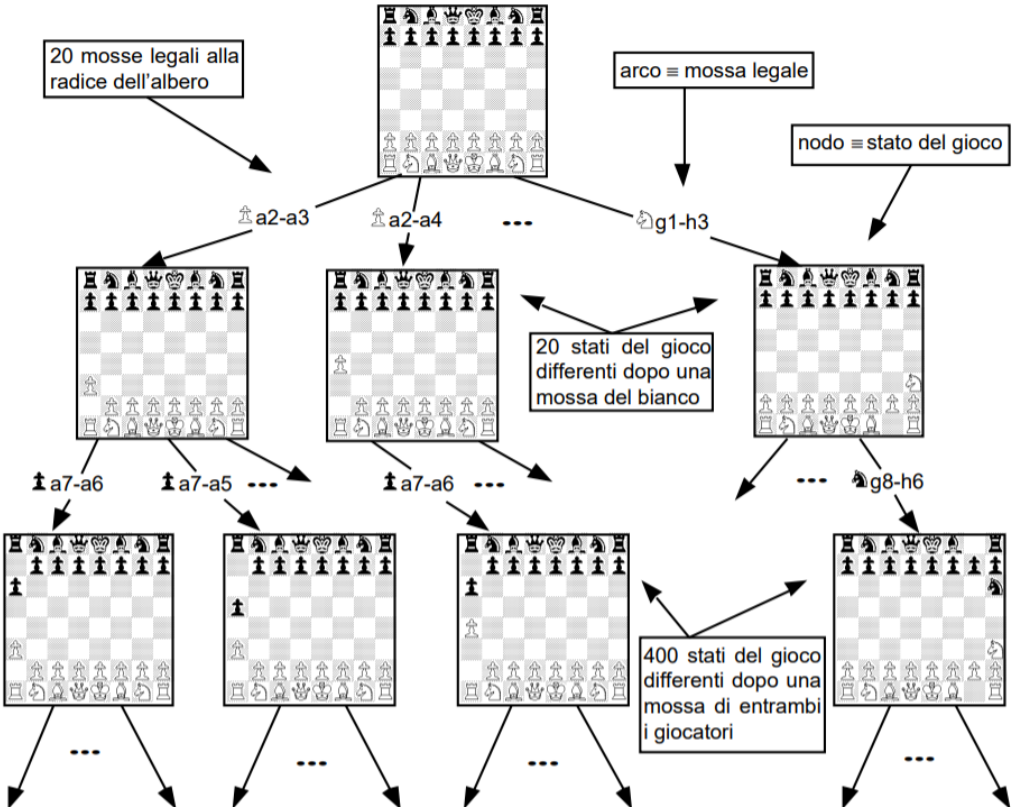


Figura 1.1: Come appare l'albero di gioco degli scacchi [18]

mentre il numero di partite possibili (sequenza di mosse) è circa 10^{120}. In media, una partita di torneo, tra giocatori umani, dura 40 mosse. Secondo le stime citate, supponendo di avere a disposizione una macchina che esegua l'algoritmo su detto, elaborando una intera variazione in un microsecondo, il calcolo della mossa migliore richiederebbe circa 10^{90} anni. É evidente che tale algoritmo, nella pratica, è tutt'altro che utile.

Questo porta a concludere che è praticamente impossibile costruire una macchina in grado di giocare una partita perfetta. D'altra parte, sarebbe banale realizzarne una che sia in grado solo di giocare correttamente, rispettando le regole. Perciò, l'obiettivo della programmazione scacchistica è di realizzare una macchina in grado di approssimare sufficientemente bene un *giocatore perfetto*. Al momento, l'obiettivo da raggiungere è di riuscire a superare la bravura dei migliori giocatori umani (detti GM, Gran Maestri).

Se svolgere un'esplorazione completa è impossibile, comunque è possibile svolgere un'esplorazione parziale dell'albero di gioco, seguendo determinati criteri di scelta e valutazione. Migliori saranno tali criteri, migliori saranno le performance della macchina. Difatti, sono riconosciuti due principali approcci:

- tipo A: esplorazione di *forza bruta*. Si valutano tutte le mosse possibili per ogni posizione, fino ad un certo livello dell'albero di gioco (effetto orizzonte). Il livello in cui ci si ferma dipende dai vincoli imposti (potenza computazionale, tempo, memoria). Oggi un calcolatore ottimizzato nell'analisi dell'albero di gioco arriva ad esplorare in media alberi profondi 12/14 semi-mosse in 3 minuti

- tipo B: mediante una certa euristica si valutano solo le mosse *promettenti*, mentre si ignorano le altre. Questo permette di giungere a

maggiori profondità nell'albero di gioco. D'altra parte, l'euristica deve essere il più accurata possibile

1.4 Cos'è un motore scacchistico?

Basandosi sulla definizione di Intelligenza Artificiale data da John McCarthy, Marvin Minsky, Allan Newell, Herbert Simon e Claude Shannon, un giocatore artificiale è un calcolatore in grado di giocare al gioco degli Scacchi e dotato di prestazioni che, ad un osservatore comune, sembrerebbero essere di pertinenza esclusiva dell'intelligenza umana. Nell'ambito della programmazione scacchistica, tale giocatore artificiale è detto *motore scacchistico*. Questi è l'entità che implementa e attua l'algoritmo di cui si è discusso alla sezione [1.3.3 - *L'albero di gioco*]. Inoltre, può essere una macchina appositamente progettata ed adibita allo scopo, oppure solo un software. Nel primo caso, ogni suo aspetto potrà essere ottimizzato al meglio.

1.4.1 Descrizione funzionale

Il requisito funzionale fondamentale è: data in input una posizione, restituire in output la mossa che viene valutata come migliore. Ciò implica che il motore sia in grado, internamente, di rappresentare, memorizzare e gestire tutte le informazioni relative allo stato attuale della partita.

Altro requisito è che il calcolo della mossa migliore venga effettuato in tempi ragionevoli, cioè equiparabili ai tempi che impiegherebbe un giocatore umano (circa 10 minuti [2]).

[2] questo valore potrebbe variare nel caso si volesse progettare un motore scacchistico da competizione

Inoltre, è necessario fornire il motore di un protocollo di comunicazione, sia per poter ricevere la posizione iniziale, sia per poter restituire la mossa calcolata. Ad oggi, si sono affermati due protocolli, ritenuti quasi uno standard: UCI [7] (Universal Chess Interaface) e XBoard [3]. Infatti, la maggior parte dei motori sono specializzati solo sulla computazione della mossa, quindi sono dei veri e propri back-end. Ogni altra funzionalità è demandata ad un componente esterno[3], con il quale si comunica con i protocolli su citati, appunto. Tale componente esterno è un vero e proprio front-end, permettendo ad un utente esterno di accedere alle funzionalità del motore. Oltre a questo, spesso si occupa anche della GUI, del controllo della validità della posizione, della legalità della mossa giocata, del controllo della regola delle tre ripetizioni e del controllo della regola delle cinquanta mosse. Si è scelto di avere questo due componenti disaccoppiati per assicurare portabilità, manutenibilità, meccanismi di testing dei motori automatizzati e infine una maggiore usabilità per l'utente finale [4].

Nelle seguenti figure sono rappresentati un'activity diagram e una sorta di communication diagram. Nel primo, è evidenziata la successione più semplice di operazioni che può compiere un motore. Nel secondo, sono rappresentate le interazioni che il motore ha con l'esterno, le quali avvengono tramite un'interfaccia (in genere grafica) di front-end; da notare che l'avversario può essere un giocatore umano, oppure un altro giocatore artificiale.

1.4.2 Descrizione strutturale

I principali componenti del motore sono:

[3]Esempi open-source sono: Arena GUI, PyChess, Tarrash, Fritz

[4]non bisogna dimenticare che alcuni motori possiedono anche un aspetto commerciale

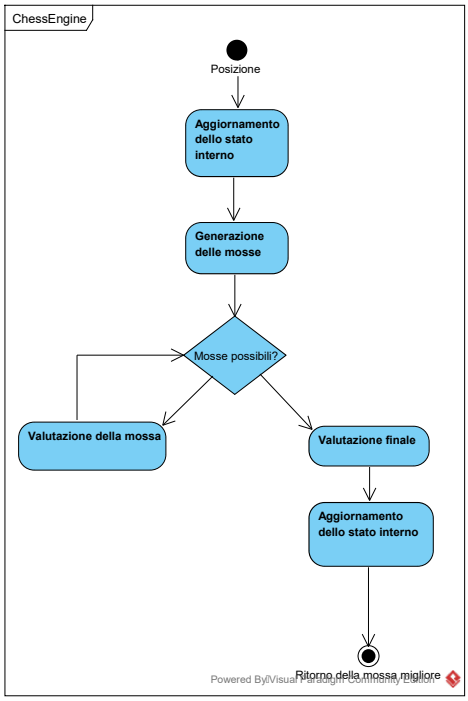


Figura 1.2: Activity diagram che evidenzia la successione più semplice di operazioni che può compiere un motore

Figura 1.3: Communication diagram che evidenzia le interazioni che il motore ha con le esterno; da notare che l'avversario può essere un giocatore umano, oppure un altro giocatore artificiale

- Stato Interno: si occupa di mantenere tutte le informazioni relative alla partita che il motore sta svolgendo.
- Generatore di mosse: data una posizione e il colore che deve muovere, genera tutte le mosse legali (o pseudo-legali) possibili
- Esploratore: svolge l'esplorazione dell'albero di gioco, supportato dal Valutatore
- Valutatore: esegue la valutazione di una posizione data, secondo definiti criteri
- Interfaccia: gestisce tutte le comunicazioni da e verso l'esterno (cioè verso il front-end), implementando un protocollo di comunicazione
- Time Manager: nel caso di motori da competizione, si occupa della gestione del tempo a disposizione. In genere, questo dato è comunicato dall'interfaccia front-end. Nel caso in cui il tempo a disposizione scadesse, verrà sollevata un'eccezione che dovrà essere opportunamente risolta

Tutti questi componenti sono fortemente accoppiati. Con facilità, si potrebbero definire delle interfacce comuni, ma queste impatterebbero notevolmente sulle prestazioni. Infatti, Stato Interno e Generatore di mosse vengono spesso implementati in un unico componente. Altrettanto può avvenire per Ricercatore e Valutatore.

1.5 Stato dell'arte

Negli anni '50, per la limitata potenza di calcolo a disposizione, fu esplorato l'approccio di tipo B. Purtroppo, non portò i risultati sperati. Mano a mano

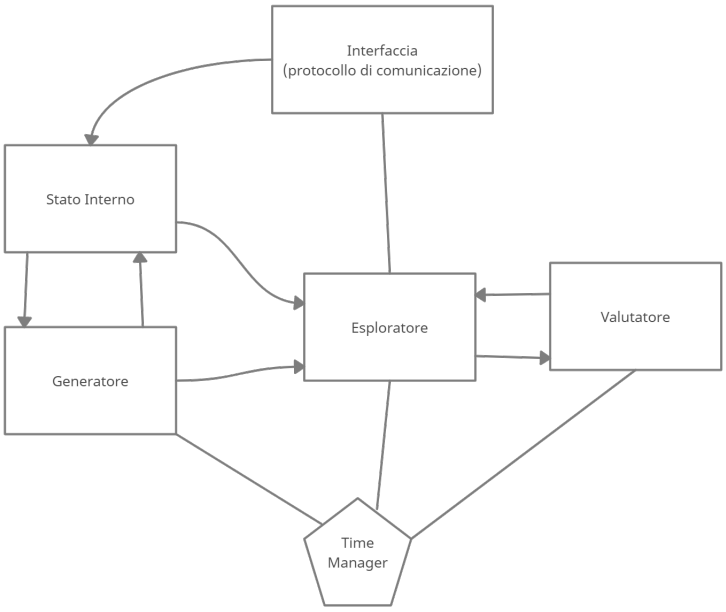


Figura 1.4: rappresentazione sommaria dei componenti su detti. Le linee e frecce indicano i flussi di dati scambiati tra tali componenti

che la potenza computazionale aumentava, ci si spostò verso l'approccio di tipo A, il quale fu portato all'esasperazione utilizzando le architetture parallele. Il coronamento di tale sforzo giunse con la vittoria di Deep Blue su Kasparov.

In seguito, a partire dagli ultimi anni '90, si cercò di mettere a punto un metodo che non fosse di sola forza bruta, emulando i giocatori umani. Infatti, questi erano superiori perché possedevano esperienza di gioco e la capacità di riconoscere facilmente motivi tattici e strategici. Con i paradigmi emergenti del Machine Learning, del Patter Recognition, degli Algoritmi Genetici e delle Reti Neurali si è cercato di catturare proprio questi aspetti, ottenendo risultati molto buoni. Infine, negli ultimi anni, l'interesse generale della comunità ha iniziato a spostarsi sulle varianti degli Scacchi, le quali presentano un'informazione incompleta (Fisher Chess e Kriegspiel). Questo

sarà il nuovo fronte della ricerca.

In conclusione, lo studio degli Scacchi ha permesso di sviluppare una notevole conoscenza e comprensione dei giochi ad informazione completa, stimolando lo studio della conoscenza in sé, dell'intelligenza e del ragionamento. Tant'è vero che molti studiosi vedono nella Teoria dei Giochi, nell'Intelligenza Artificiale e nella Programmazione Scacchistica una anticipazione di quella che sarà una materia molto più ampia, la quale Donald Michie [5] definisce *Ingegneria della Conoscenza*.

[5] ex direttore del Department of Machine Intelligence and Perception dell'Università di Edinburgo

Capitolo 2

Rappresentazione e Generazione

2.1 Cosa rappresentare?

Per poter svolgere le proprie funzionalità in maniera efficace ed efficiente, un motore scacchistico deve possedere, al suo interno, una rappresentazione completa dello stato del gioco:

- posizione attuale
- posizioni passate (necessarie per la regola delle tre ripetizioni)
- mosse effettuate
- numero di mosse intercorse dall'ultima cattura o movimento di pedone (necessario per la regola delle 50 mosse)
- stato della partita (apertura/medio-gioco/finale)

2.2 Rappresentazione della scacchiera e dei pezzi

Una posizione è caratterizzata da:

- pezzi in gioco (colore e posizione sulla scacchiera)
- colore attivo (quale colore ha il tratto[1])
- cronologia delle mosse effettuate (necessario per la gestione del meccanismo make/undo move)
- diritti d'arrocco (chi può eseguire quale tipo di arrocco)
- possibilità di cattura en passant

Individuare una rappresentazione corretta è relativamente semplice; invece, è più complesso individuarne una efficiente. Vi sono due principali approcci:

- orientamento alla scacchiera: si modella l'associazione: *una determinata casa è occupata da un pezzo oppure è vuota?*
- orientamento ai pezzi: si modella l'associazione: *un determinato pezzo quale casa occupa?*

2.2.1 Rappresentazione orientata alla scacchiera

Questa rappresentazione è stata storicamente la prima ad essere concepita. In questa, l'entità centrale è la casa di una scacchiera, la quale può contenere un unico pezzo, oppure essere vuota. Il pezzo è rappresentato in maniera

[1] tratto, in gergo scacchistico, indica il colore a cui spetta muovere

semplice, in genere attraverso un singolo valore. É un approccio semplice, che porta ad implementare anche un generatore di mosse semplice; d'altra parte potrebbe generare inefficienze: durante la partita, al più 32 celle risulteranno occupate, mentre le altre saranno sicuramente vuote.

Essendo lo scacchiera un quadrato 8x8, è immediata l'associazione ad una matrice 8x8, di conseguenza, se (in genere) gli elementi contenuti in una matrice sono numeri, allora si fa corrispondere un valore numerico ad ogni pezzo (si veda la figura d'esempio).

Bianco	Tipo pezzo	Nero
1	Pedone	-1
2	Cavallo	-2
3	Alfiere	-3
4	Torre	-4
5	Regina	-5
6	Re	-6

-4	-2	-3	-5	-6	-3	-2	-4
-1	-1	-1	-1	-1	-1	-1	-1
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1
4	2	3	5	6	3	2	4

Questa rappresentazione è tanto semplice ed immediata quanto inefficiente: per sfruttare al massimo il meccanismo della gerarchia della memoria, i moderni calcolatori memorizzano gli array in forma di una lista lineare; di conseguenza il compilatore necessita di una moltiplicazione e una addizione per accedere ad un elemento di un array bidimensionale. Basti pensare che l'ordine del numero di accessi allo stato della scacchiera durante la ricerca della mossa migliore è circa $10^5 \sim 10^6$, quindi è fondamentale ottimizzare tale operazione.

Il passo successivo è di utilizzare un array monodimensionale di 64 elementi. In questo caso, dimezzate le operazioni necessarie all'accesso, emerge

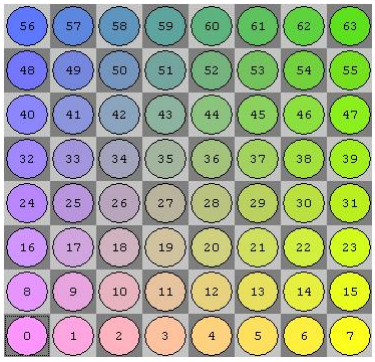


Figura 2.1: Per ogni casa, è indicato l'indice che la identifica. Fonte: ChessProgramming Wiki

il problema della gestione dei movimenti sulla scacchiera. Ogni tipo di pezzo può muoversi in modo differente, quindi sono necessari meccanismi di gestione personalizzati. Questi meccanismi necessitano l'esecuzione di vari controlli, i quali richiedono più accessi all'array stesso. La gestione degli indici non validi (cioè delle case non esistenti) diventa complessa. Tali fattori determinano l'inefficienza di questo approccio.

Per risolvere tale problema, è stata proposta la rappresentazione 12x10. Oltre alle 64 case, vengono aggiunte delle *case sentinelle* attorno alla scacchiera. Definita un'associazione biunivoca tra indici e case, durante la generazione delle mosse, se viene generato un *indice sentinella*, l'attuale mossa in corso di generazione verrà scartata e si passerà alla generazione della successiva. Si veda la figura 2.2.

Altro approccio, sviluppato in contemporanea al precedente, è il 0x88. Si utilizza un array di 128 elementi, distinguendo tra case valide e case non valide (si veda la figura 2.3). Come nel caso di 12x10, dopo aver definito un'associazione biunivoca tra indici e case, nel momento in cui viene generato un indice la cui corrispondente casa sia non valida, la mossa verrà scartata. Il criterio di distinzione tra case valide e invalide è il seguente: dato un indice, la

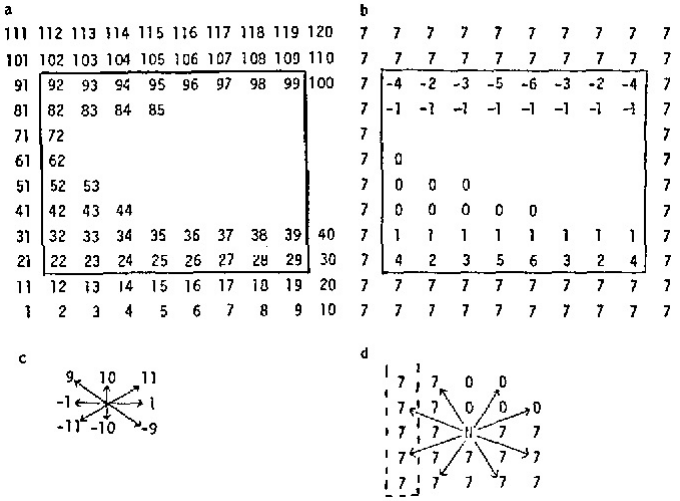


Figura 2.2: Esempio di rappresentazione 12x10 nel motore COKO III [11]. Viene presentato l'esempio di movimento di un cavallo, ai margini della scacchiera.

corrispondente casa è valida se l'operazione di maschera, con il valore binario 1000.1000, restituisce un risultato uguale a zero.

2.2.2 Rappresentazione orientata ai pezzi

L'entità centrale è il pezzo, il quale si trova in gioco (occupando una casa) o fuori gioco (ad esempio è stato catturato o rimosso dopo la promozione). Quindi viene mantenuta una struttura dati contenente i pezzi ancora in gioco.

Un pezzo è caratterizzato da:

- tipo (pedone, torre, cavallo, alfiere, donna, re)
- colore (bianco, nero)
- stato (in gioco, fuori gioco)
- casa in cui si trova

8	70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	7F
7	60	61	62	63	64	65	66	67	68	69	6A	6B	6C	6D	6E	6F
6	50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F
5	40	41	42	43	44	45	46	47	48	49	5A	5B	5C	5D	5E	5F
4	30	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F
3	20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F
2	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
1	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
	A	B	C	D	E	F	G	H								

Figura 2.3: Rappresentazione della scacchiera secondo 0x88. Gli indici invalidi sono quelli presenti nelle celle più scure. Gli indici sono presentati in notazione esadecimale

Queste informazioni sono state rappresentate in molti modi diversi. Il più semplice è assegnare un valore numerico ad ogni tipo di pezzo, come mostrato nella tabella 2.2.1

Un approccio più economico potrebbe essere di assegnare ad ogni pezzo un valore di 8 bit. Infatti, bisogna essere in grado di distinguere tra 6 tipi di pezzi e 2 differenti colori. In questo modo si occupa meno memoria e si potranno sfruttare le operazioni logiche; d'altra parte il codice sarà più complesso.

L'approccio più moderno è stato l'utilizzo della tecnologia ad oggetti. Da un punto di vista di manutenibilità e leggibilità del codice sarebbe l'approccio migliore, ma non garantisce le prestazioni delle Bitboards.

2.2.3 Bitboards

É la rappresentazione che ha portato i migliori risultati ottenuti fin'ora, diventando la più utilizzata. L'idea di base consiste nell'analogia di *grandezza* che vi è tra una scacchiera (64 case) e una parola di 64 bit: ogni bit potrebbe

0000.0000.0000.0000.0000.0000.0000.0000.0000.0000.0000.0000.1111.1111.0000.0000

A8 H8 A1 H1

Figura 2.4: Esempio di bitboard associata ai pedoni bianchi nella posizione iniziale di una partita

8	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0
2	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	0	0
	A	B	C	D	E	F	G	H

Figura 2.5: Corrispondenza tra la bitboard rappresentata nella figura 2.5 e la notazione algebrica

rappresentare lo stato di occupazione di una determinata cella. Quindi, si associa una parola di 64 bit ad ogni tipo di pezzo di un certo colore, ottenendo la posizione di ogni pezzo in gioco.

Le sole bitboards associate ai tipi di pezzi non permettono di ricostruire univocamente una posizione. Infatti, è necessario mantenere bitboards per: tutti i pezzi in gioco, tutti i pezzi neri, tutti i pezzi bianchi e per ogni tipo di pezzo di un certo colore. Un numero maggiore di bitboards permette ulteriori ottimizzazioni durante la generazione delle mosse, ma bisogna fare attenzione, poiché è necessario mantenerle tutte consistenti.

Questa rappresentazione permette di sfruttare al meglio l'odierna architettura dei calcolatori a 64 bit: si lavora a livello di bit e con le operazioni logiche, cioè le più veloci messe a disposizione. Un altro vantaggio consiste nel *parallelismo sui bit* [2]: in alcune situazioni, possono venir generate più

[2]Termine coniato da R. Hyatt [4]

mosse con una sola operazione. Ad esempio, nella posizione di partenza di una partita, essendo tutti i pedoni allineati, è possibile generare per tutti la mossa di avanzamento con una sola operazione.

Le bitboards permettono una notevole ottimizzazione. D'altra parte, aumentano la complessità del codice, il quale diventa meno leggibile, testabile e manutenibile. Altro aspetto negativo, è la necessità di mantenere consistenti tutte le bitboards presenti in memoria, in qualunque istante.

2.3 Generazione delle mosse

2.3.1 Rappresentazione delle mosse

Le informazioni che caratterizzano una mossa sono:

- casa di partenza
- casa di arrivo
- se si tratta di una cattura e l'eventuale pezzo catturato
- se si tratta di una promozione e l'eventuale pezzo in cui si promuove
- se si tratta di un arrocco

Gli approcci per rappresentare queste informazioni sono molti. In generale, la rappresentazione delle mosse è legata alla rappresentazione della scacchiera e dei pezzi, oltre che alle caratteristiche del generatore delle mosse. Più informazioni verranno rappresentate/memorizzate e migliori potranno essere le ottimizzazioni attuabili, anche se a scapito della memoria. É necessario memorizzare quali mosse vengono applicate alla scacchiera per il meccanismo di *make-undo move*[3].

[3]Il meccanismo make-undo move verrà trattato nella sezione 3.2

2.3.2 Generazione delle mosse

Vi sono tre tipi di generatore:

- selettivo: esegue un analisi sulla posizione e secondo determinati criteri, genera solo alcune delle possibili mosse
- incrementale: genera un numero limitato di mosse, poi attende che gli venga richiesto di generarne altre
- completo: genera ogni mossa possibile

Il generatore selettivo è stato il primo ad essere studiato, permettendo di attuare la strategia di tipo B (vedi 1.3.3): si voleva spostare la complessità della valutazione di una buona mossa dal componente di esplorazione dell'albero al generatore; purtroppo, non sono stati raggiunti i risultati desiderati. Il generatore completo potrebbe apparire come il più semplice da realizzare, ma spende molto tempo nella generazione di mosse le quali si riveleranno pessime. Invece, il generatore incrementale, pur essendo più complesso, permette di attuare varie ottimizzazioni.

Altro aspetto da considerare è il tipo di mossa generata:

- generazione delle mosse legali: vengono generate solo le mosse legali. Questo implica spendere del tempo per verificare la legalità (ad esempio non bisogna lasciare il re in presa, oppure non si può catturare un pezzo amico)
- generazione delle mosse pseudo-legali: per ragioni di efficienza, la legalità della mossa viene controllata in una seconda fase (in genere, durante l'esplorazione)

Ad oggi, i risultati migliori sono stati ottenuti dai generatori incrementali di mosse pseudo-legali.

2.3.3 Vettori di Movimento

Si supponga di utilizzare una rappresentazione orientata alla scacchiera: si ha un array monodimensionale e vi è una corrispondenza diretta tra indici e case. Per ogni pezzo è possibile definire dei vettori di movimento: relazioni matematiche che legano l'insieme di tutti gli indici di partenza, con l'insieme di tutti i possibili indici d'arrivo. Ad esempio, chiamando k l'indice della casa di partenza, per il re è possibile definire:

```
k - 9, k + 1, k + 11, k - 10, k + 10, k - 11, k - 1, k + 9
```

5						
4	24	25	26	27	28	
3	16	17	18	19	20	
2	8	9	10	11	12	
1	0	1	2	3	4	
	A	B	C	D	E	F

Esempio grafico dei vettori di movimento del re, con riferimento all'immagine2.2.1, a pagina 22. Il re è posizionato nella casa rossa e potrebbe muoversi sulle case gialle.

Definiti opportunamente tutti i vettori di movimento per ciascun pezzo, il generatore non dovrà far altro che applicare tali formule all'attuale posizione. Il vantaggio di questo metodo è l'assoluta semplicità.

2.3.4 Mappe dei Movimenti

Per ogni tipo di pezzo p e per ogni casa di partenza, si definisce una bitboard in cui il generico bit i può assumere i valori:

$$i = \begin{cases} 1 & \text{se } p \text{ può muoversi sulla casa di arrivo corrispondente al bit } i \\ 0 & \text{altrimenti} \end{cases}$$

8	0	0	0	0	1	0	0	0
7	0	0	0	0	1	0	0	0
6	0	0	0	0	1	0	0	0
5	0	0	0	0	1	0	0	0
4	1	1	1	1	0	1	1	1
3	0	0	0	0	1	0	0	0
2	0	0	0	0	1	0	0	0
1	0	0	0	0	1	0	0	0
	A	B	C	D	E	F	G	H

Figura 2.6: Un esempio di bitboard di movimento per la torre, dalla casa di partenza e4

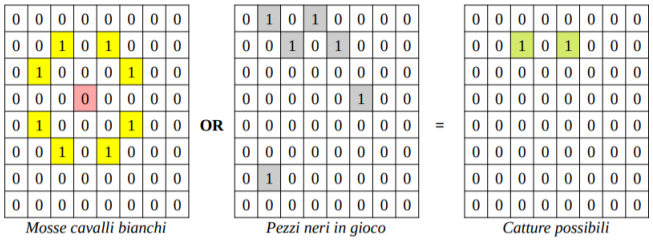


Figura 2.7: Nell'esempio si mostra come sia possibile ricavare le mosse di cattura per un cavallo bianco posizionato nella casa d5

Grazie alla logica binaria, è possibile definire una serie di operazioni tra le bitboards di rappresentazione e di movimento, ottenendo informazioni sulla posizione e generando tutte le mosse pseudo-legali.

Le migliori performance vengono ottenute utilizzando un grande *dizionario*. La chiave è data dal tipo di pezzo e dalla casa di partenza. Il valore associato è dato dalla bitboard contenente le possibili case d'arrivo. Definito tale dizionario, il generatore non farebbe altro che eseguire una ricerca al suo interno, estraendo la bitboard di cui necessita. Da tale bitboard potrà ricavare tutte le informazioni necessarie alla generazione delle mosse.

La generazione dei movimenti di re, pedoni e cavalli risulta essere piuttosto semplice. Invece, è più complessa la gestione dei pezzi che scorrono

[4], in quanto questi non possono "scavalcare" gli altri pezzi fermi sulla loro strada. Questo implica dei controlli particolari. Inoltre, l'arrocco e le catture en passant vengono gestite separatamente. Come già detto, le bitboards rendono il codice più performante e flessibile, ma anche più complesso; inoltre il *dizionario* sfrutta una notevole quantità di memoria.

[4]torre, alfiere, regina

Capitolo 3

Ricerca

3.1 Qual è lo scopo della ricerca?

Il gioco degli scacchi è un gioco di decisioni. *Decidere* implica essere in grado di analizzare, comprendere e simulare l'evoluzione della scacchiera nel tempo. Di svolgere tali operazioni, se ne occupano i componenti denominati Esploratore e Valutatore [1].

Come esposto nella sezione 1.3.3, l'Esploratore ideale è in grado di simulare l'intera partita, dalla posizione data fino alla conclusione. Nella pratica, ciò non è possibile, quindi bisogna tentare con approcci alternativi.

Il primo approccio è di fornire al motore un dizionario di suggerimenti, il quale avrebbe come chiave una determinata posizione e come valore la miglior mossa rispettiva da giocare. Per il numero di posizioni possibili nel gioco degli scacchi, questo approccio si è rivelato utilizzabile solo nella fasi di apertura e di finale, dove necessitano di essere mantenute un numero limitato di posizioni. Il secondo approccio è di mettere a punto un metodo di esplorazione parziale dell'albero di gioco: prevedere criteri di selezione dei

[1] vedi 1.4.2 a pagina 15

rami e di valutazione dei nodi. Con altre parole, bisogna definire un algoritmo di ricerca, il quale data una posizione iniziale, individui qual è il percorso (la successione di mosse) che conduca alla migliore posizione possibile per il tratto a cui spettava la mossa.

3.2 Make-Undo Move

Nell'albero di gioco degli scacchi, ad ogni nodo corrisponde una posizione, la quale è caratterizzata da una serie di informazioni (disposizione dei pezzi in gioco, tratto attivo, cronologia delle mosse effettuate, diritti d'arrocco e possibili catture en passant). Memorizzare la disposizione dei pezzi per ogni nodo diventa un'operazione molto dispendiosa: è necessario eseguire molte copie e non si avrebbe memoria sufficiente a disposizione.

Quindi, non tutti i nodi (posizioni) sono memorizzati in maniera completa, cioè con la precisa disposizione dei pezzi sulla scacchiera. Infatti, è sufficiente mantenere la sequenza di mosse applicate a partire dalla posizione iniziale. Viene mantenuto solo un numero ristretto di posizioni complete (a volte solo la posizione iniziale) e su queste il motore effettua le proprie analisi e valutazioni.

Data una posizione iniziale P, se al motore interessa valutare un certo discendente $D1$ di P, applicherà la sequenza di mosse $D1.mosse$. Conclusa la valutazione, scorrendo a ritroso $D1.mosse$, annullerà le mosse applicate ripristinando lo stato iniziale di P. Poi proseguirà con l'analisi di un successivo discendente, procedendo come già descritto.

Nel seguito di questo capitolo, verranno esposti gli algoritmi di ricerca più importanti che sono stati utilizzati nella programmazione scacchistica.

3.3 Minimax e Alphabeta

3.3.1 Minimax

Questo algoritmo discende direttamente dal teorema del Minimax di von Neumann [15], il quale afferma:

Teorema 1 (del Minimax)**.** *In ogni gioco a somma zero, tra due giocatori e con un numero finito di strategie, esistono un valore V e una strategia per ciascun giocatore tali che:*

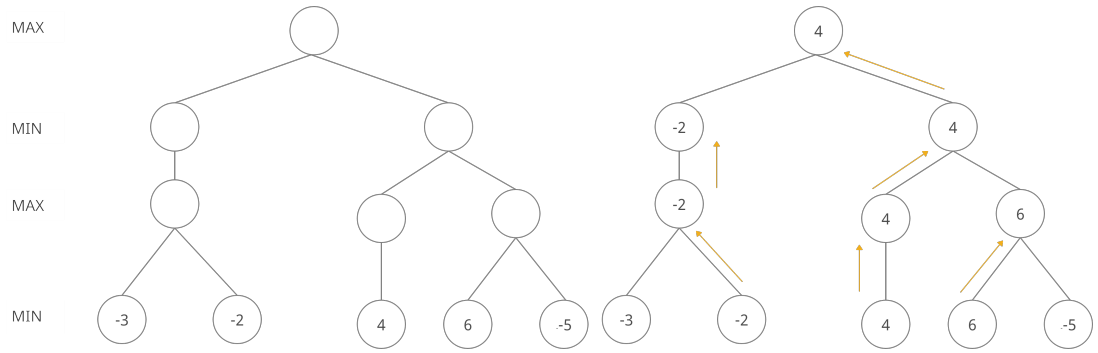
1. *data la strategia del giocatore 2, il miglior payoff possibile per il giocatore 1 è V*
2. *data la strategia del giocatore 1, il miglior payoff possibile per il giocatore 2 è $-V$*

In sostanza, il guadagno di un giocatore corrisponde alla perdita dell'altro e viceversa. Infatti, l'algoritmo Minimax (Algorithm 1) sceglie ogni mossa in modo da ridurre al minimo il massimo dei vantaggi che l'avversario potrebbe conseguire: esplora tutti i possibili nodi dell'albero fino alla profondità limite (*effetto orizzonte*) e mediante una funzione di valutazione misura la bontà effettiva della posizione a cui si è giunti. Per convenzione, il primo giocatore che deve muovere è detto Max (massimizzante), l'altro Min (minimizzante). I valori di $+\infty$ e $-\infty$ verranno adottati per le mosse che portano allo vittoria rispettivamente Max o Min. Il valore di ritorno sarà il valore del nodo iniziale, così da poter determinare qual è il figlio più conveniente in cui *spostarsi* e la corrispondente mossa. La funzione CalcolaValore(*nodo*) rappresenta il servizio offerto dal Valutatore all'Esploratore; la discussione delle operazioni svolte da tale funzione sarà oggetto del prossimo capitolo.

Di seguito vi è un esempio di applicazione:

Algorithm 1 Algoritmo Minimax

```
function MINIMAX(nodo, prondità)
    if nodo è una foglia OR profondità = 0 then
        return CALCOLAVALORE(nodo)
    end if
    if tocca all'avversario then
        α ← ∞
        for ogni figlio di nodo do
            α ← min (α, MINIMAX(figlio, profondità − 1))
        end for
    else                                                    ▷ tocca a noi
        α ← −∞
        for ogni figlio di nodo do
            α ← max (α, MINIMAX(figlio, profondità − 1))
        end for
    end if
    return α
end function
```



Quest'algoritmo rappresenta l'approccio di tipo A (forza bruta): maggiore sarà la profondità e maggiore sarà l'accuratezza. Al momento, tutti i dati raccolti confermano ciò: un motore in grado di analizzare un numero considerevolmente maggiore di nodi, rispetto ad un altro, vince sistematicamente.

Una possibile semplificazione del minimax è Negamax (Algorithm 2), il quale sfrutta la seguente caratteristica dei giochi a somma zero: $max(a, b) = -min(-a, -b)$. Nel nostro caso, il valore della posizione di un giocatore è la negazione del valore dell'altro. Di conseguenza, nel percorso di propagazione dei valori verso la radice non è necessario distinguere tra Max e Min: è sufficiente assegnare al nodo padre il massimo dei valori dei figli cambiati di segno.

Questi due algoritmi sono relativamente semplici, ma assolutamente inefficienti. Infatti, la complessità è $O(b^d)$, dove b corrisponde al fattore di branching e d alla profondità a cui si vuole arrivare.

3.3.2 Effetto orizzonte

La ricerca si conclude sempre ad una fissata profondità. Questo implica che si potrebbe interrompere l'analisi nel bel mezzo di una sequenza di scambi o

Algorithm 2 Algoritmo Negamax

```
1: function NEGAMAX(nodo, prondità)
2:     if nodo è una foglia OR profondità = 0 then
3:         return CALCOLAVALORE(nodo)
4:     end if
5:     max ← −∞
6:     for ogni figlio di nodo do
7:         valore ← −NEGAMAX(figlio, prodondità − 1)
8:         if valore > max then
9:             max ← valore
10:        end if
11:    end for
12:    return max
13: end function
```

di un movimento tattico: la valutazione della posizione sarebbe falsata dal temporaneo sbilanciamento di materiale.

Queste posizioni sono dette quiescenti e riguardano, appunto, scambi di materiale e motivi tattici. Il problema viene risolto attuando delle ricerche quiescenti: quando si giunge alla massima profondità d'esplorazione, se la posizione è quiescente, allora viene invocata una ricerca circoscritta al suo sotto-albero, con profondità contenuta.

3.3.3 Alphabeta

Come visto nella sezione precedente, Minimax e Negmax attuano un approccio di forza bruta. Purtroppo, si può facilmente mettere in evidenza che molti nodi (posizioni) non contribuiscono alla valutazione della radice (ad

esempio, si tratta di mosse assolutamente stupide, come lasciare in presa la propria regina). L'idea è di non sprecare risorse nella visita di tali nodi e dei rispettivi sotto-alberi eseguendo la potatura (o taglio) del ramo.

Nella versione di base di Alphabeta, si esegue una ricerca in profondità (depth-first) da sinistra a destra. Sempre supponendo che Max sia il primo giocatore a muovere (giocatore massimizzante), si definiscono:

- α è il punteggio minimo che Max può raggiungere; viene inizializzato a $-\infty$. Durante il calcolo, rappresenta il valore della peggiore mossa possibile attualmente calcolata per Max.

- β è il punteggio massimo che Min può raggiungere; viene inizializzato a $+\infty$. Durante il calcolo, coincide con il valore della migliore mossa possibile attualmente calcolata per Min.

α e β verranno propagati opportunamente dai nodi padre ai nodi figli e dai nodi figli ai nodi padre. Sono possibili tre tipi di tagli:

- taglio alpha-beta: avviene se $\alpha > \beta$

- taglio alpha: avviene quando, dato un nodo N con mossa al giocatore Min, il genitore di N ha un valore provvisorio maggiore del valore di N stesso

- taglio beta: avviene quando, dato un nodo N con mossa al giocatore Max, il genitore di N ha un valore provvisorio minore del valore di N stesso

Algorithm 3 Algoritmo Alphabeta

```
function AlphabetaMax(nodo, α, β, prodondità)
    if nodo è una foglia OR profondità = 0 then
        return CalcolaValore(nodo)
    end if
    for ogni figlio di nodo do
        valore ← AlphabetaMin(nodo, α, β, profondità − 1)
        if valore ≥ β then
            return β                                    ▷ taglio beta
        end if
        if valore > α then
            α ← valore
        end if
    end for
    return α
end function
function AlphabetaMin(nodo, α, β, prodondità)
    if nodo è una foglia OR profondità = 0 then
        return CalcolaValore(nodo)
    end if
    for ogni figlio di nodo do
        valore ← AlphabetaMax(nodo, α, β, profondità − 1)
        if valore ≤ α then
            return α                                    ▷ taglio alpha
        end if
        if valore < β then
            α ← valore
        end if
    end for
    return β
end function
```

3.3.4 Efficienza di AlphaBeta

Seguendo la classificazione proposta T.A. Marsland e F. Popovic [13], dato un albero di gioco soggetto a ricerca Alphabeta, si possono distinguere i seguenti tipi di nodi:

- nodi PV: compongono la continuazione principale (principal variation), cioè il primo cammino (dalla radice ad un nodo terminale/orizzonte) che viene esaminato nella ricerca
- nodi CUT: rappresentano alternative ai nodi PV. Sono gli unici nodi in cui possono avvenire tagli
- nodi ALL: sono nodi successori dei nodi CUT. A loro volta i successori dei nodi ALL sono ancora di tipo CUT.

Mentre tutti i sotto-alberi dei nodi PV e ALL sono sempre esplorati, a causa dei tagli, solamente un sottoinsieme dei sotto-alberi dei nodi CUT sarà preso in considerazione. Nel caso ottimale di ricerca Alphabeta, il primo discendente diretto di ogni nodo CUT determina un taglio. L'albero esplorato in questa situazione particolare viene detto albero minimale. Esso definisce la porzione minima dell'albero di gioco che verrà comunque e in ogni modo sempre esplorata da tale algoritmo.

Per confrontare le prestazioni di minimax e alphabeta, si considera un albero di gioco uniforme di fattore di diramazione b e profondità d [8]. Nel caso medio, la complessità è: $O(b^{(d/2)})$. Invece, nel caso peggiore, si ha la stessa complessità di minimax: $O(b^d)$. Da notare che la prestazione ottimale è ottenuta quando, ad ogni livello, viene esplorato solo il primo figlio e si taglia tutti gli altri. In questo caso, l'albero si dice *perfettamente ordinato*, poiché la mossa migliore è stata analizzata per prima. In generale, un albero di gioco

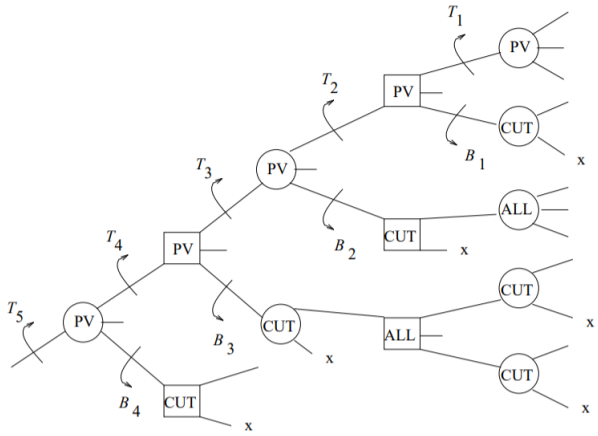


Figura 3.1: raffigurazione dei tipi di nodi in un albero soggetto a ricerca Alphabeta

è ordinato se la mossa migliore in un generico nodo è fra le prime ad essere esaminata. Invece, si genera il caso peggiore quando si ha l'ordinamento inverso delle mosse: la mossa migliore sarà l'ultima ad essere esaminata, quindi non avverranno tagli. Di fatto, l'efficienza dell'algoritmo Alphabeta è tanto maggiore quanto più forte è l'ordinamento dell'albero, essendo la differenza fra le dimensioni dell'albero minimale e massimale notevoli.

Di seguito verranno esposte alcune tecniche euristiche per il miglioramento di Alphabeta.

3.3.5 Ordinamento delle mosse

Come già detto, un ordinamento più forte porta più tagli e quindi alla costruzione di un albero molto più vicino a quello minimale. Non esiste un criterio esatto di valutazione delle mosse. Infatti l'unico modo per poter conoscerne il valore, sarebbe di effettuare un'esplorazione completa dell'albero di gioco. Di conseguenza, ci si affida a criteri euristici.

I principali approcci sono:

- valutazione statica della posizione. Si ordinano le mosse da quella che porta alla posizione di miglior vantaggio fino a quella che porta alla posizione di peggior svantaggio

- tipologia della mossa. Si ricalca il tatticismo delle mosse forzanti: vengono prima valutati eventuali scacchi al re avversario, poi eventuali catture, poi gli arrocchi, poi le promozioni e infine tutte le altre tipologie di mosse

- utilizzo delle tabelle (History, Transposition Table, Refutation Table). Le posizioni già presenti nella tabella e con un punteggio elevato, saranno analizzate prima delle altre mosse. L'idea è che se una mossa, fino ad ora, si è rivelata buona in posizioni simili, continuerà a rivelarsi buona anche per posizioni simili che si potrebbero ancora incontrare

Infine, l'ordinamento delle mosse si è rivelato essere importante anche per gli algoritmi alternativi ad Alphabeta (vedi sezione 3.4).

3.3.6 Aspiration Search

Maggiore sarà il numero dei tagli, minore sarà il numero di nodi esplorati, avvicinandosi sempre più all'albero minimo. Durante la ricerca, la probabilità di operare un taglio è tanto maggiore quanto più stringente è l'intervallo fra i parametri α e β. La versione base di Alphabeta inizializza la finestra di ricerca con: $(-\infty, +\infty)$. Invece, Aspiration Search la inizializza con: $(V - err, V + err)$, dove V è il valore minimax stimato della posizione data, con un certo margine d'errore err.

É possibile che il valore stimato sia errato e che il valore calcolato esca dalla finestra di aspirazione. Nel caso in cui tale valore sia maggiore di $V + err$, allora è stata effettuata una sottostima; questo non provoca alcun

problema. Invece, nel caso in cui tale valore sia minore di $V - err$, allora è stata effettuata una sovrastima, la quale non è accettabile: bisogna ripetere la ricerca con una finestra corretta.

L'idea su esposta è portata al limite dalla Minimal Window Search, dove la finestra viene impostata sui valori $(V, V + \delta)$, dove δ è scelto il più vicino possibile a V. L'applicazione dell'algoritmo Alphabeta con finestra minimale ad un albero ordinato prende il nome di Principal Variation Search

Nel caso in cui $\delta = 0$, allora si parla di Null Window Search: non si avrà una vera e propria finestra ma solo un valore di confronto, V, che potrà assumere il ruolo, in taluni casi, di vincolo superiore, mentre in altri di vincolo inferiore. Questo approccio porta molti tagli; di contro, è necessario eseguire più volte la ricerca, *aggiustando*, di volta in volta, il valore di V.

3.3.7 History Heuristic

L'idea è che una mossa che risulta la migliore in una posizione, con molta probabilità, è altrettanto valida in altre posizioni (nelle quali è legale). In particolare la tabella History memorizza, per una mossa m, il numero di posizioni visitate nelle quali m è stata riconosciuta come migliore oppure abbia causato un taglio. Tale conteggio costituisce una misura della probabilità che una certa mossa risulti la migliore in posizioni future. Il numero di ingressi della tabella è dato da tutte le possibili combinazioni di casa di partenza e casa di arrivo per ciascun colore (in totale 8196). Risultati sperimentali hanno mostrato che l'utilizzo di questa tabella permette di ottenere un buon ordinamento [16] e dei buoni miglioramenti di Aplhabeta.

3.3.8 Null Move Heuristic

L'idea è: data una posizione, supponendo di non muovere (passare la mossa all'avversario, permettendogli di giocare due mosse consecutive), se la situazione finale non risulta essere a nostro estremo sfavore, allora noi eravamo in vantaggio. Formalmente, il giocatore che ha il tratto passa e si attua una ricerca Alphabeta per l'avversario. Se tale ricerca ritorna un valore V minore di *alpha*, allora verrà ritornato V, perché, appunto, rappresenta un nodo talmente favorevole che probabilmente determinerà un taglio.

Seppur molto semplice, è un metodo rischioso e che potrebbe portare molti problemi in sequenze tattiche di gioco. In linea di massima, è assolutamente sconsigliato usare questa tecnica nella fase finale del gioco e quando si è sotto scacco (si andrebbero a generare solo mosse illegali).

3.3.9 Selective Extension e Selective Reduction

L'idea è di approfondire l'analisi di rami che si ritengono promettenti e non approfondire quei rami che, invece, si ritengono non promettenti. Alcuni esempi di Extensions possono essere:

- se il giocatore che ha il tratto è sotto scacco, estendi l'analisi di una mossa
- se il giocatore che ha il tratto ha appena effettuato una cattura, estendi l'analisi di n mosse
- se un pedone è stato promosso, estendi l'analisi di n mosse

Alcuni esempi di Reduction possono essere:

- ignora la mossa, se viene catturato un pezzo tra torre e regina
- ignora la mossa, se un pezzo viene inchiodato al re

Figura 3.2: Esempio di trasposizione. Infatti, questa posizione può essere ottenuta applicando la sequenza ***1. d4 Nf6 2. Bg5 e6 3. e4 Be7 4. Nc3 d5 5. e5 Nfd7 6. Bxe7 Qxe7 7. f4****; oppure la sequenza* ***1. e4 e6 2. d4 d5 3. Nc3 Nf6 4. Bg5 Be7 5. e5 Nfd7 6. Bxe7 Qxe7 7. f4***

3.3.10 Tabella delle Trasposizioni

L'intero spazio di ricerca del gioco degli Scacchi può essere rappresentato come un grafo, dove i nodi sono le posizioni possibili e gli archi le mosse legali. Lungo tale grafo, si può mostrare che in alcuni casi, dato un nodo di partenza e uno di arrivo, sono possibili più percorsi. Questo fenomeno è detto Trasposizione. Nella seguente figura 3.2 viene presentato un esempio.

Questo fenomeno è assolutamente indesiderato. Infatti, durante la ricerca, si analizza più volte la stessa posizione a profondità differenti causando uno spreco di risorse. Inoltre, per la frequenza con la quale avvengono le trasposizioni durante una ricerca, è un fenomeno che non può essere trascurato.

Per risolvere il problema, l'idea è di memorizzare il risultato dell'analisi effettuata su di una posizione; nel momento in cui, durante la ricerca, tale posizione sarà incontrata nuovamente, allora non si esplorerà il rispettivo sotto-albero, ma si ritornerà il valore già memorizzato. In altre parole,

durante la ricerca, viene costruito un dizionario delle posizioni già valutate (tabella delle trasposizioni) e ad ogni posizione che si deve analizzare si eseguirà una ricerca in tale dizionario: se la posizione è presente, si utilizza il valore già calcolato, altrimenti si prosegue con l'analisi e poi con l'opportuno aggiornamento del dizionario.

Tale tabella, per essere effettivamente efficiente, è implementata come Hash Table. Su quali informazioni vadano memorizzate, la comunità non è d'accordo: vi sono vari approcci e risultati sperimentali discordanti. Inoltre, l'implementazione della tabella è fortemente dipendente dall'Esploratore e dal Valutatore: deve essere tutto ottimizzato, altrimenti non si riuscirà ad ottenere un effettivo beneficio dall'applicazione di questa tecnica. Comunque, le informazioni essenziali che devono essere presenti nella tabella sono:

- chiave
- profondità
- valore
- identificatore della posizione (la chiave non è sufficiente ad identificare univocamente ogni possibile posizione, perché la stessa chiave potrebbe venire associata a più posizioni)

Invece, le seguenti informazioni sono presenti o meno in base alle scelte di design e ottimizzazione:

- mossa migliore
- età (tempo di permanenza nella tabella). Quest'informazione è utilizzata per attuare alcune politiche di gestione delle collisioni. In generale, si preferisce mantenere una posizione individuata ad una profondità inferiore, poiché permetterà di generare più tagli.

- tipo di nodo (PV, CUT, ALL)

A fronte dell'alto numero di posizioni possibili, è inevitabile che avvenga una collisione. Le politiche di gestione delle collisioni sono innumerevoli e non verranno approfondite in questa sede. Ovviamente, tutti i meccanismi ideati hanno la caratteristica comune di essere semplici e veloci, altrimenti si andrebbero ad annullare i benefici assicurati dalla tabella.

3.3.11 Hashing di Zobrist

Non è l'unica, ma indubbiamente è la funzione di hashing più utilizzata nella programmazione scacchistica. Questa tecnica ha una formulazione più generale [19], ma in questa sede verrà trattata solo la sua applicazione al gioco degli Scacchi. La popolarità di tale tecnica è dovuta a due fattori:

- posizioni molto simili producono chiavi molto diverse. Questo consente di avere una buona distribuzione dei valori: le collisioni sono relativamente limitate rispetto all'utilizzo di memoria

- la chiave di una posizione può essere costruita a partire da una già nota (un vantaggio che si abbina perfettamente al meccanismo di make/undo move)

Infatti si esegue il calcolo completo della chiave solo per la posizione iniziale. Dopodiché, in base alla mossa applicata, si aggiornerà opportunamente la chiave per ottenere quella della nuova posizione, con un dispendio minimo di risorse. La dimensione della chiave è di 64 bit: è stato il miglior compromesso trovato tra il bisogno di ridurre al minimo le collisioni e l'occupazione di memoria.

La generazione delle chiavi si basa su un insieme di array i cui elementi saranno degli interi pseudo-casuali (per un totale di 781 numeri):

- un array per ogni tipo di pezzo in ogni casa: *zScacchiera[colore][tipo pezzo][casa occupata]*
- un valore che indichi se il tratto che deve muovere è Nero (*zTratto*)
- un array per ogni tipo di arrocco che può essere realizzato (*zDirittiArrocco[colore][tipo arrocco]*)
- un array per le colonne della scacchiera, per il controllo della possibilità di cattura en passant (*zColonne[colonna]*)

Si veda l'algoritmo4 per un esempio di pseudo codice.

Algorithm 4 Calcolo della chiave di Zobrist

```
1:  function CalcolaChiaveZobrist(posizione)
2:      chiave ← 0
3:      if muove Nero then
4:          chiave ← chiave ⊕ zTratto
5:      end if
6:      for ogni pezzo in gioco do
7:          chiave ← chiave ⊕ zScacchiera[colore][tipopezzo][casaoccupata]
8:      end for
9:      for ogni diritto d'arrocco attivo do
10:         chiave ← chiave ⊕ zDirittiArrocco[colore][tipoarrocco]
11:     end for
12:     for ogni colonna in cui è possibile una cattura en-passant do
13:         chiave ← chiave ⊕ zColonne[colonna]
14:     end for
15:     return chiave
16: end function
```

Per quanto riguarda l'aggiornamento, si sfrutta la proprietà di idempotenza dell'operazione di XOR. Si veda l'algoritmo 5 per un esempio di pseudo-codice.

Algorithm 5 Aggiornamento della chiave di Zobrist

```
function AggiornaChiaveZobrist(chiave, posizione, cronologiamosse)
    nuovaChiave ← chiave
    if cambiato il Tratto then
        nuovaChiave ← nuovaChiave ⊕ zTratto
    end if
    for ogni pezzo che ha subito una variazione nell'ultima mossa do ▷ pezzo mosso, pezzo catturato, pedone promosso, pezzo promosso
        nuovaChiave ← chiave⊕zScacchiera[colore][tipopezzo][casadipartenza]
        nuovaChiave ← chiave⊕zScacchiera[colore][tipopezzo][casadiarrivo]
    end for
    for ogni diritto d'arrocco che ha subito una variazione do
        nuovaChiave ← nuovaChiave ⊕ zDirittiArrocco[colore][tipoarrocco]
    end for
    for ogni colonna in cui è variata la possibilità di cattura en-passant do
        nuovaChiave ← nuovaChiave ⊕ zColonne[colonna]
    end for
    return chiave
end function
```

3.3.12 Lazy Evaluation

L'idea è di puntare il più possibile sulla forza bruta e riuscire ad esplorare il maggior numero di nodi: più nodi si visiteranno e maggiore sarà la probabilità di aver individuato una buona mossa. Tale ricerca si appoggierà ad una funzione di valutazione semplice, veloce e approssimativa (non interessa il valore preciso). Nonostante l'apparente semplicità, questo approccio nasconde delle insidie: se la funzione di valutazione risulta essere eccessivamente approssimativa (nel caso peggiore, viene eseguita una sovrastima del valore delle posizioni), la ricerca risulterà falsata.

3.4 Approcci alternativi

In questa sezione verranno presentati velocemente i principali algoritmi sviluppati in alternativa ad Alphabeta.

3.4.1 Iterative Deepening

In generale, Iterative Deepening [10] è un algoritmo di ricerca in un grafo, ma è stato rielaborato per l'applicazione al gioco degli Scacchi. Esso costituisce un approccio misto tra la ricerca in profondità (depth-first) e la ricerca in larghezza (breadth-first). Esegue una ricerche AlphaBeta di profondità crescente: viene eseguita una visita completa dei nodi a profondità 1, poi a profondità 2, poi a profondità 3 e così via. Si veda l'algoritmo 6 per un esempio di pseudo-codice.

Il primo vantaggio offerto da questo algoritmo è che possibile limitare il tempo di ricerca: nel momento in cui scade il tempo limite, verrà ritornata la mossa migliore trovata all'ultimo livello esplorato in maniera completa. Altro vantaggio è la possibilità di effettuare maggiori controlli e analisi, poiché è

Algorithm 6 Algoritmo Iterative Deepening

1: **function** ITERTIVEDEEPENING($posizione, maxprofondità$)
2: $profonditàRicerca \leftarrow 1$
3: $valore \leftarrow 0$
4: **for** $d \leq maxprofondità$ **do**
5: $valore \leftarrow$ ALPHABETA($nodo, -\infty, +\infty, profonditàRicerca$)
6: ORDINAMOSSE ▷ le mosse sono riordinate in modo che la migliore sia visitata per prima nella successiva iterazione
7: **end for**
8: **return** $valore$
9: **end function**

possibile confrontare tutte le posizioni che sono situate alla stessa profondità. Il terzo vantaggio, forse il più importante, è il riempimento efficiente delle tabelle euristiche già viste (History, Trasposition) e il miglioramento dell'Aspiration Window (è possibile stimare con più precisione i valori limite). Ciò permette di effettuare un numero considerevole di tagli quando avviene l'invocazione di Alphabeta. Un ulteriore incremento delle prestazioni può essere ottenuto effettuando l'ordinamento delle mosse.

Da notare che quando si passa ad una profondità $d+1$, i nodi alle profondità inferiori saranno nuovamente visitati. Questo potrebbe apparire come un effetto indesiderato e come uno spreco di risorse. Eppure non è un problema, anzi l'algoritmo è molto efficiente. Infatti, in un albero di gioco degli scacchi, il tempo necessario all'esplorazione del nuovo livello $d+1$ domina il tempo di esplorazione dell'albero dal livello 0 al livello d. Questo è dovuto al numero delle foglie-orizzonte (nodi del livello $d+1$, dove la ricerca si ferma) che è considerevolmente maggiore rispetto al numero dei nodi intermedi.

3.4.2 Numeri di Cospirazione

Si tratta di una ricerca probabilistica. Definito l'insieme dei possibili valori che possono essere assunti da un generico nodo, a ciascuno di questi valori viene assegnata una probabilità che ne esprime la plausibilità. Durante l'esplorazione dell'albero, in base ai risultati ottenuti, è possibile che alcune probabilità vengano modificate. Viene fissata una probabilità di soglia: superarla implica che è poco probabile che il valore corrispondente sia il valore effettivo del nodo. L'aspetto interessante di questo approccio, è che la generazione di tali probabilità è indipendente dalla specifica applicazione.

La grandezza che descrive tale probabilità è il numero di cospiratori [14]. Un cospiratore è un nodo il quale, se cambiasse valore (ad esempio lo si espande e si esegue una valutazione più precisa), determinerebbe il cambiamento del valore di un proprio antenato. Il numero di cospiratori di un dato nodo V è definito come il numero minimo di foglie cospiratrici necessarie a determinare un cambiamento del valore di V. In altre parole, ci si chiede quanto è probabile che si stia sottostimando o sovrastimando il valore effettivo di V. Inoltre, è definita una soglia di cospirazione CT: dato un nodo ed un possibile valore v, se il corrispondente numero di cospirazione è superiore a CT, allora tale valore è improbabile che sia il valore effettivo.

Dato un nodo che non è ancora stato espanso, con valore stimato v (al momento è visto dall'algoritmo come una foglia-orizzonte) questi avrà numero di cospiratori 0 associato a v, invece ogni altro valore che potrebbe assumere avrà un numero di cospiratori pari a 1. Per quanto riguarda le foglie effettive dell'albero (posizione di scaccomatto o stallo), queste avranno un valore assoluto f con numero di cospiratori 0, mentre tutti gli altri possibili valori avranno associato un numero di cospiratori pari a ∞.

L'algoritmo procede nell'esplorazione dell'albero di gioco, fino a che non

ha aumentato tutti i valori possibili della radice oltre la CT, tranne uno, il quale verrà ritornato come il valore più probabile. Il valore ritornato non è esatto; comunque più è alta la soglia di cospirazione e maggiore sarà la plausibilità del valore di ritorno. Per ulteriori dettagli, si rimanda il lettore all'articolo [12].

Quest'approccio si è rivelato essere performante quanto AlphaBeta; anzi, in posizioni in cui è necessario attuare tatticismi e mosse forzanti, si comporta meglio. Questo è proprio dovuto al fatto che i rami che attuano queste tattiche, presentano meno cospiratori: l'algoritmo esplorerà a fondo questi rami facendo salire in fretta tutte le soglie di cospirazione della radice. Tale aspetto è la forza, ma allo stesso tempo la debolezza di questo algoritmo: nelle posizioni in cui non è possibile attuare dei tatticismi, vengono esplorati a fondo dei rami senza però riuscire ad innalzare il numero dei cospiratori per la radice. Per questo motivo, il tempo di convergenza dell'algoritmo potrebbe essere alto, se non infinito [12].

3.4.3 Monte Carlo

Anche questo approccio svolge una ricerca probabilistica. L'albero di gioco viene costruito in maniera incrementale e asimmetrica. Ad ogni iterazione dell'algoritmo, viene applicata una politica di albero, la quale individua quale sarà il nodo che si andrà ad analizzare durante l'iterazione corrente. Questa esegue considerazioni di esplorazione (quali aree dell'albero sono state esplorate e quali no) e di guadagno (quali aree dell'albero appaiono promettenti). A partire dal nodo selezionato, viene svolta una simulazione (si esplora una sequenza di nodi scelti casualmente fino a che non si verifica una fissata condizione che determina la terminazione della simulazione stessa). In base ai

risultati ottenuti, l'albero verrà aggiornato, cioè si propagheranno i risultati dal nodo appena esplorato ai suoi antenati.

Questo algoritmo deriva dall'omonimo metodo statistico, nato nell'ambito della fisica statistica. Si è mostrato essere adatto alla risoluzione di problemi in molti campi differenti. I risultati più sbalorditivi che ha permesso di raggiungere sono stati raggiunti nello sviluppo di AlphaZero, l'attuale miglior giocatore articiale di Go. Visti i risultati negli altri ambiti, anche nella programmazione scacchistica si sta iniziando ad utilizzare questo approccio, promettendo interessanti risultati in un futuro prossimo. Per ulteriori dettagli, si rimanda il lettore all'articolo [1].

3.4.4 Parallelizzazione

É possibile attuare differenti tipi di parallelizzazione; i principali sono:

- parallelismo nella valutazione di un nodo: il calcolo di ogni termine della funzione di valutazione viene assegnato ad un processore differente. Il guadagno di prestazione dipende dalla scomponibilità della funzione stessa

- ricerca con finestre parallele: ogni processore attua una ricerca Alpha-beta con finestra di aspirazione differente. Il guadagno di prestazione è relativamente limitato

- mapping tra spazio di ricerca e processori: attraverso una funzione di hashing si associano insiemi di nodi ad un determinato processore. Il guadagno di prestazione è relativamente buono, ma comunque limitato dall'overhead dovuto al coordinamento necessario tra i processori

- decomposizione dell'albero di gioco: ad ogni processore, è assegnata l'esplorazione di un determinato sotto-albero distinto. Questo approc-

cio è il più complesso: sono necessari un coordinatore (che si occupi di attuare la decomposizione) e un meccanismo di comunicazione tra i processori. Il guadagno di prestazione è relativamente buono, ma limitato dall'algoritmo di decomposizione scelto e dall'overhead necessario per il coordinamento dei processori

Capitolo 4

Valutazione

4.1 Qual è lo scopo della valutazione?

In questa sezione si esporranno quali sono i criteri e le tecniche che permettono ad un motore scacchistico di comprendere se una data posizione è a proprio favore. La maggior parte dell'*intelligenza* di un motore risiede proprio nel componente chiamato Valutatore, il quale implementa la funzione di valutazione: è l'unico a possedere una serie di conoscenze tattiche e strategiche relative al gioco, le quali permettono di stimare l'attuale andamento della partita.

Le conoscenze che si implementano derivano dalla conoscenza umana degli Scacchi, maturata nel corso di secoli (tra partite e studi analitici). Il processo di analisi ed elaborazione che viene svolto da un giocatore umano si basa sull'occhio e sul cervello: il lavoro svolto in sinergia da questi due organi è incredibilmente efficiente. Purtroppo, non siamo ancora di grado di spiegare appieno tale meccanismo, per cui, i motori scacchistici non possono che tentare l'emulazione di tali processi.

4.2 La Funzione di Valutazione

La Funzione di Valutazione è una funzione che associa ad una posizione un numero reale:

$$\mathbf{f} : posizione \rightarrow \Re \tag{4.1}$$

L'approccio più utilizzato è il seguente, detto *simmetrico*:

$$f = \begin{cases} +\infty & \text{vittoria per Bianco} \\ x & x > 0 \text{ , vantaggio per Bianco} \\ x & x = 0, \text{ parità} \\ x & x < 0, \text{ vantaggio per Nero} \\ -\infty & \text{vittoria per Nero} \end{cases}$$

La funzione di valutazione cerca di catturare i due più importanti aspetti di una posizione:

- materiale
- posizione

Quindi, è possibile esprimerla:

$$f = materialeBianco + posizioneBianco + materialeNero + posizioneNero$$

Sono anche state implementate funzioni di valutazione che ritornano più di un valore, ma al momento non si sono dimostrate migliori.

4.2.1 Materiale

Ogni tipo di pezzo ha un valore. Di seguito alcuni esempi:

Scala di valori	♙	♘	♗	♖	♕	♔
Scala di Shannon [17]	100	300	300	500	900	non definito
Scala di Crafty [5]	100	325	325	500	1050	4000
Scala di Stockfish (medio-gioco) [6]	126	781	825	1276	2538	non definito
Scala di Stockfish (finale) [6]	208	854	915	1380	2682	non definito

I valori su mostrati derivano sia da modelli probabilistici che da prove sperimentali. In generale, il valore di un pezzo non è assoluto, ma è strettamente legato alla posizione presente sulla scacchiera. Inoltre, possono essere eseguite alcune considerazioni riguardanti un insieme pezzi, ad esempio:

- bonus per il possesso della coppia degli alfieri (questi, se assieme, assicurano il controllo sia delle diagonali chiare, che delle diagonali scure)
- penalità per non avere nessun pedone (sono una risorsa molto importante nel finale, poiché possono venir promossi)

Quindi:

$$materialeBianco = \sum_{i \in I} valorePezzo_i + bonus + penalità$$

Dove I è l'insieme degli indici dei pezzi presenti in gioco. É definito analogamente $materialeNero$.

4.2.2 Posizione

Nella letteratura scacchistica, il concetto di Posizione è articolato e profondo e non vi è una definizione che riesca a cogliere tutto in un unico periodo. Però, è possibile affermare che la posizione riguarda questi aspetti del gioco:

- sviluppo: molti pezzi non sono particolarmente efficaci nella loro posizione di partenza, ma lo sono in determinate case della scacchiera, dove bisogna cercare di posizionarli il prima possibile e al contempo impedire all'avversario di effettuare il proprio sviluppo

- controllo della scacchiera: controllare le case centrali assicura la possibilità di poter muovere più agevolmente i propri pezzi e quindi di svolgere attacchi più veloci ed efficaci.

- mobilità: i pezzi pesanti hanno bisogno di "spazio" e non dovrebbero essere lasciati dietro ai pedoni o comunque ostruiti da altri pezzi.

- sicurezza del re: il valore del re è inestimabile, quindi deve essere posizionato velocemente dietro agli altri pezzi (in genere con l'arrocco)

- struttura pedonale: i pedoni costituiscono la prima linea di attacco e di difesa, permettendo di attuare i punti su detti.

Un giocatore umano è in grado di soppesare con relativa facilità questi aspetti, ma non è affatto semplice formalizzarli e rappresentarli all'interno di un motore scacchistico. L'approccio più semplice è ridurre tutto a delle condizioni booleane facilmente verificabili. Se una condizione viene rispettata, allora si riceverà un bonus, altrimenti si riceverà una penalità. Nel seguito, vengono presentati alcuni esempi dal punto di vista di Bianco (è possibile definire queste condizioni *simmetricamente* per Nero):

- se i miei pedoni occupano il centro (esempio di controllo)

- se il mio re è stato arroccato (esempio di sicurezza del re)

- se i miei cavalli occupano il centro (esempio di sviluppo)

- se non vi è alcun pedone isolato (esempio di struttura pedonale)

Quindi:

$$posizioneBianco = \sum_{j \in J} valore_j$$

dove J è l'insieme degli indici corrispondenti ai bonus/penalità rilevati. Analogamente per Nero.

Approcci più complessi prevedono la costruzione di tabelle e grafi, con appunto, l'idea di riuscire a rappresentare il *legame* che vi è tra i pezzi, i quali hanno un valore relativo, dipendente da più parametri: pezzi presenti gioco e fase del gioco ecc... .

4.3 Considerazioni

Come si è detto, la funzione di valutazione offre una stima e non un valore esatto. In teoria, avere a disposizione una funzione più precisa porterebbe la ricerca a produrre una scelta migliore. Purtroppo, una funzione più precisa implica un numero maggiore di operazioni ed elaborazioni da dover effettuare sulla posizione analizzata e quindi risorse "tolte" alla ricerca. Al contrario, una funzione più semplice, seppur più approssimativa, è più veloce e richiede meno risorse. In definitiva, risulta necessario trovare il giusto equilibrio: alcuni algoritmi di ricerca necessitano di funzioni di valutazione più raffinate (ad esempio Alphabeta), ad altri è sufficiente una stima.

Altro punto da sottolineare è che la funzione di valutazione definita in questa sezione è una valutazione statica, cioè riguarda un preciso momento del gioco. Questa racchiude le informazioni che riguardano il presente, non quelle che riguardano il passato e (soprattutto) il futuro. Ciò limita la capacità di individuare tattiche e strategie. A questo problema, il primo approccio è stato di dotare i motori di un database di sequenze/pattern, quindi, come per i giocatori umani, dotare il motore di una *esperienza di gioco*. I

risultati sono stati discordanti e non abbastanza soddisfacenti. Di recente, si sta cercando di risolvere il problema attraverso algoritmi genetici, machine learning e reti neurali.

Capitolo 5

Progetto personale

5.1 Qual è lo scopo del mio progetto personale?

Durante l'attività di tirocinio interno, ho sviluppato un semplice motore scacchistico. Tale motore (sostanzialmente un prototipo) è stato concepito per mostrare e mettere in pratica i concetti teorici esposti in questo elaborato, nonché le idee più significative che hanno caratterizzato l'evoluzione della programmazione scacchistica. Infine, il prototipo non è particolarmente performante, ma è funzionante ed adibito a futuri miglioramenti.

5.2 Requisiti da soddisfare

Requisiti e vincoli di progetto sono:

- requisiti funzionali già esposti nella sezione Descrizione funzionale (vedi 1.4.1)
- tempi di sviluppo limitati (3 mesi)

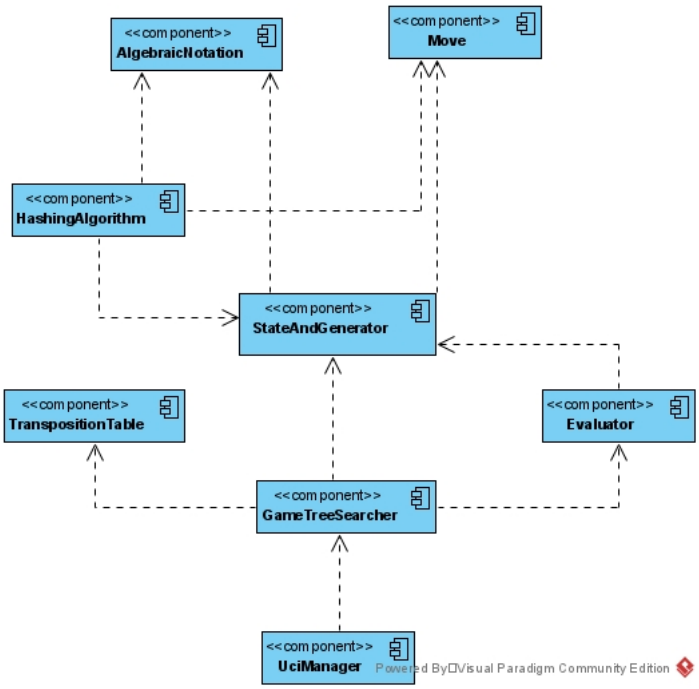


Figura 5.1: Component diagram che rappresenta l'architettura del motore realizzato

- implementazione del protocollo UCI, per assicurare un minimo di portabilità e facilitare la fase di testing
- codice chiaro e manutenibile

Per i requisiti su esposti, è stato scelto di implementare il motore interamente in Python. Inoltre, il motore *conosce* tutte le regole degli scacchi tranne la regola delle 3 ripetizioni e la regola delle 50 mosse: la verifica è demandata al front-end.

5.3 Panoramica sull'architettura

L'architettura del motore è raffigurata nella figura 5.1.

AlgebraicNotation è il modulo dedicato alla gestione delle coordinate algebriche , così da permettere una manipolazione più agevole dei vettori di movimento, oltre alla semplificazione di una serie di controlli sulla legalità delle mosse.

Move è il modulo dedicato alla gestione delle mosse e dei diritti di arrocco.

HashingAlgorithm è il modulo che implementa gli algoritmi di hashing, in particolare quello di Zobrist.

StateAndGenerator è il modulo che si occupa di mantenere lo stato interno e della generazione delle mosse, offrendo tutta una serie di servizi a *GameTreeSearcher* e *Evaluator*.

Evaluator è il modulo che implementa le funzioni di valutazione.

TransposizionTable è il modulo che implementa la gestione della Tabella di Trasposizione.

GameTreeSearcher è il modulo che implementa gli algoritmi di ricerca

UciManager è il modulo che gestisce le comunicazioni da/verso l'esterno, implementando il protocollo UCI. All'accensione di motore, questo modulo costituisce il punto di partenza dell'esecuzione. Dopo aver ricevuto dall'esterno l'opportuna sequenza di comandi UCI, avvierà la ricerca attraverso un Python-Thread.

5.4 Rappresentazione e generazione

Sono stati realizzati due differenti moduli di rappresentazione dello stato interno e di generazione delle mosse.

5.4.1 Rappresentazione a oggetti e generatore legale

Il primo modulo implementa: una rappresentazione ad oggetti orientata ai pezzi; un generatore legale e incrementale, il quale sfrutta i vettori di movimento.

L'entità centrale è la classe RealPiece, la quale rappresenta un pezzo in gioco. RealPiece mantiene tutta una serie di informazioni sul pezzo (casa che occupa, colore, re alleato e altro). La lista dei pezzi presenti in gioco è incapsulata nell'entità ListPiece, la quale offre vari servizi agli altri componenti del motore; i più importanti sono il meccanismo di make/undo move e di cronologia delle mosse effettuate durante la partita. Tale classe è specializzata da ListPieceHashValue, la quale offre anche i servizi di gestione del valore di hashing associato alla posizione.

Ogni pezzo incapsula la logica necessaria alla generazione delle proprie mosse legali, in particolare i propri vettori di movimento. Inoltre, la sottoclasse del Re implementa un metodo apposito di verifica di scacco a se stesso. Il concetto stesso di mossa assume la forma di una classe mantenendo le informazioni seguenti: casa di partenza, casa di arrivo, pezzi coinvolti, tratto che ha mosso, tipo di mossa (movimento, cattura, promozione, arrocco ecc...) e se vi è un eventuale scacco al re avversario.

5.4.2 Rappresentazione ibrida e generatore pseudo-legale

Il secondo modulo implementa: una rappresentazione orientata ai pezzi che si ispira al concetto di bitboard; un generatore pseudo-legale e incrementale, il quale sfrutta le mappe di movimento.

Vengono mantenute delle liste *generali* (case occupate, case occupate da pezzi bianchi, case occupate da pezzi neri, en passant, arrocchi possibili, case

che permettono la promozione); e delle liste *specifiche* per ogni tipo di pezzo e colore (case occupate dai pedoni bianchi, case occupate dai pedoni neri e così via per ogni tipo di pezzo...). Effettuando opportune operazioni tra queste liste, è possibile ottenere ogni informazione necessaria alla generazione delle mosse e alla gestione dello stato del gioco.

Per ogni tipo di pezzo che non scorre (re, cavallo, pedone), vi è un dizionario dei movimenti avente come chiave la casa in cui si trova il pezzo e come valore la lista delle case in cui potrebbe muoversi. Nel particolare, i pedoni presentano dizionari specifici (distinti per ogni colore) riguardanti anche il doppio passo, la cattura e la promozione.

Invece, è stato necessario trattare in maniera particolare i pezzi che scorrono (torre, alfiere, regina), poiché non possono scavalcare altri pezzi nell'avanzare lungo una direzione. Per quanto riguarda torre e alfiere, i loro dizionari hanno come chiave la casa di partenza e come valore una lista di quattro sottoliste. Ciascuna sottolista contiene le case che il pezzo potrebbe raggiungere avanzando in una determinata direzione (in totale, quattro direzioni, appunto). Tali case sono ordinate in maniera tale da permettere opportuni controlli che impediscano di scorrere oltre al primo ostacolo incontrato, che sia un pezzo alleato o nemico. La regina sfrutta sia il dizionario della torre che il dizionario dell'alfiere.

Le mosse generate sono pseudo-legali: si controlla solo di non catturare un pezzo alleato, mentre si verifica che il re alleato non sia in presa solo in un secondo momento, dopo l'effettiva applicazione della mossa, durante la fase di esplorazione dell'albero di gioco.

Infine, il modulo espone un'interfaccia ListPiece, la quale mette a disposizione i servizi di gestione dello stato interno e di generazione delle mosse verso gli altri componenti; di questi, i più importanti sono il meccanismo di

make/undo move, di cronologia delle mosse effettuate durante la partita e il controllo di re sotto scacco.

5.5 Ricerca

L'Esploratore implementa Minimax, Alphabeta e Iterative Deepening. Oltre alla versione base, gli ultimi due sono stati ampliati per poter supportare l'ordinamento delle mosse e l'utilizzo della Tabella di Trasposizione. In tutti i casi, ogni algoritmo è decentralizzato nella chiamata a Max (si suppone essere sempre Bianco) e nella chiamata a Min (si suppone essere sempre Nero).

Le entità centrali di questo componente sono le classi GamePosition e AlgorithmPosition. La prima rappresenta una posizione (nel senso esposto nella sezione 2.2 *Rappresentazione della scacchiera e dei pezzi*). La seconda rappresenta l'algoritmo di ricerca applicato ad una determinata GamePosition. A sua volta, GamePosition si specializza nella posizione Bianca e nella posizione Nera. Poi, per ciascun colore, vi è l'ulteriore specializzazione in GamePositionTable, cioè una posizione che supporta la gestione di un valore di hashing e della Tabella di Trasposizione (implementata in un modulo dedicato).

L'Esploratore prevede che il componente di interfacciamento con l'esterno (UciInModule) esegua un opportuno settaggio delle variabili globali (quale algoritmo applicare e se permettere l'utilizzo della tabella di trasposizione). Poi si aspetta l'assegnamento della *rootPosition*, cioè la posizione iniziale da cui si dovrà ricavare la mossa migliore.

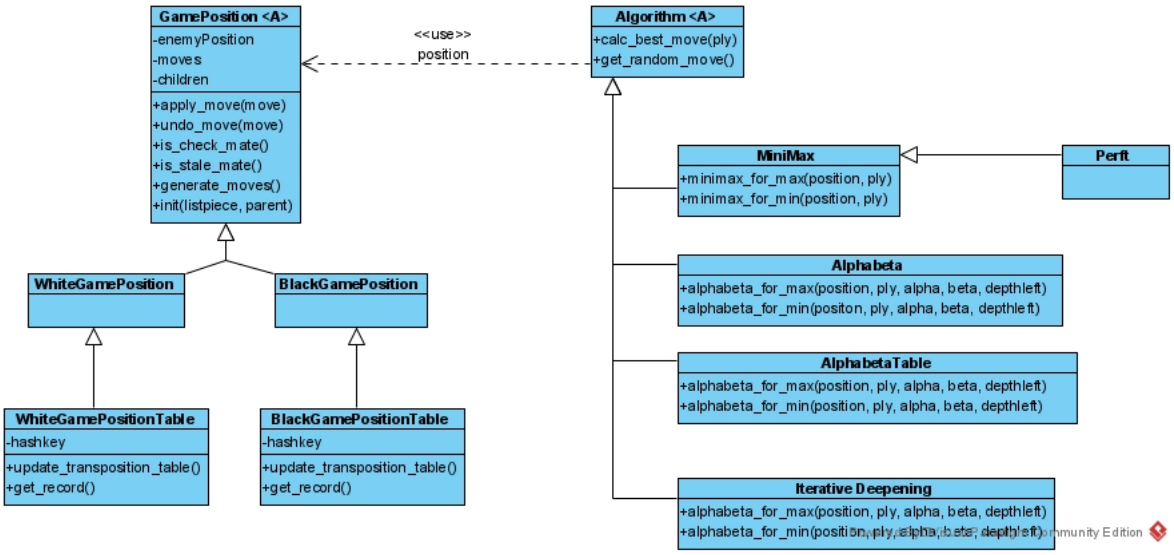


Figura 5.2: Class Diagram che rappresenta la struttura fondamentale dell'Esploratore implementato

5.6 Valutazione

Il Valutatore implementa due funzioni di valutazione. La prima è molto semplice ed esegue considerazioni solo sul materiale (Lazy Function). La seconda esegue delle considerazioni su materiale e posizione. Entrambe utilizzano i seguenti valori:

♙	♘	♗	♖	♕	♔
1	3	3	5	9	1000

La seconda funzione di valutazione cerca di catturare il concetto di valore relativo di un pezzo rispetto ad una posizione. Infatti, non vengono utilizzati i valori assoluti dei pezzi (come descritto nella sezione 4.2 Funzione di Valutazione), ma dei valori ricavati dall'applicazione di una serie di tabelle e di coefficienti ottenuti da analisi posizionali approssimate.

5.7 Debugging e testing

A causa dell'alto numero di mosse legali che possono essere generare in ogni posizione e per l'altissimo numero di nodi dell'albero di gioco, il testing di Generatore ed Esploratore risulta essere molto complesso. In definitiva, gli approcci veramente efficaci sono: controllo manuale dei risultati; far giocare al motore migliaia di partite. Infatti, le regole degli Scacchi possono apparire relativamente semplici, ma non è così banale trasformarle in condizioni booleane elementari, soprattutto nel controllo di scacco al re.

Il test più utilizzato è Perft (Performance Test o Move Path Enumeration). Fissata una profondità massima, si contano tutte le foglie-orizzonte raggiungibili a partire dalla posizione di partenza del gioco. Ad esempio, Perft(1) conta tutte le foglie-orizzonte raggiungibili dopo una mossa di Bianco; invece, Perft(4) conta tutte le foglie-orizzonte raggiungibili dopo 4 mosse. Nel corso della ricerca, grazie al contributo di ciascun progetto, si è giunti a stilare una tabella di valori ritenuti esatti. Questo test permette di comprendere a quale profondità dell'albero avviene un errore di generazione delle mosse, aiutando a circoscrivere il numero di mosse da dover controllare. Al momento, si è sicuri dei risultati fino a profondità 14. Si veda la tabella seguente [2]:

Profondità	Conteggio
0	1
1	20
2	400
3	8902
4	197281
5	4865609
6	119060324
7	3195901860
8	84998978956
9	2439530234167
10	69352859712417
11	2097651003696806
12	62854969236701747
13	1981066775000396239
14	61885021521585529237
15	2015099950053364471960

Inoltre, sono state stilate varie raccolte di posizioni più o meno complesse che dovrebbero aiutare a stimare l'*intelligenza* del motore, confrontandosi con i risultati già ottenuti dai centinai di motori testati e funzionanti. Un esempio è il Bratko-Kopec Test [9], il quale ho utilizzato proprio per testare il mio progetto.

5.8 Risultati

Qui esporrò tutti i risultati ottenuti dai test effettuati sul motore, con lo scopo di fornire un riscontro numerico che supporti la trattazione teorica. Tutti i

test sono stati effettuati sulle posizioni suggerite dal Bratko-Kopec Test e su varie altre posizioni più semplici, oltre che sulla posizione di partenza del gioco.

5.8.1 Confronto fra i generatori

I due generatori hanno prestazioni molto differenti tra loro. Il generatore legale calcola ogni mossa on-demand, dovendo svolgere più operazioni di controllo ed elaborazione delle informazioni. Invece, il generatore pseudo-legale esegue una ricerca nel dizionario idoneo, per poi eseguire controlli molto semplici. La maggior parte di tali controlli consistono nella verifica di appartenenza di un elemento alle varie liste mantenute nello stato interno. La superiorità del secondo generatore è netta e mostra quale fu l'entità del miglioramento che le bitboard introdussero.

Generatore	N. mosse al secondo
Generatore ad oggetti	174
Generatore a liste	4422

Dai test effettuati, il Generatore è risultato essere il collo di bottiglia del sistema. Nella seguente tabella, sono indicate le percentuali di tempo che il motore, in media, spende nelle varie fasi della ricerca.

Algoritmo	Tempo di generazione	Tempo di valutazione	Tempo di esplorazione
Alphabeta	50	1	49
Iterative Deepening	60	5	35

5.8.2 Confronto fra gli algoritmi

Minimax è risultato essere assolutamente inefficiente. Invece, Alphabeta con ordinamento delle mosse e Iterative Deepening hanno avuto un buon comportamento. É necessario sottolineare che l'efficienza di questi due algoritmi emerge al crescere della profondità massima di esplorazione. Infatti, lo sforzo necessario per effettuare gli ordinamenti delle mosse porta un effettivo beneficio solo se avvengono corpose potature dell'albero. Nel caso del motore sviluppato, sembra iniziare ad essere così da profondità maggiori o uguali a quattro.

Dai test eseguiti, Alphabeta e Iterative Deepening hanno mostrato tempi totali di ricerca molto simili. É interessante notare che Iterative Deepening, pur eseguendo un numero medio di tagli inferiore ad Alphabeta, grazie ai suoi ordinamenti alla fine di ogni *iterazione*, tiene testa ad Alphabeta. Nella tabella sono indicate le percentuali medie dei nodi di tipo CUT esplorati durante una ricerca rispetto al numero totale di nodi generati.

Algoritmo	Nodi CUT alpha	Nodi CUT beta
Alphabeta	$14,7$	$5,1$
Iterative Deepening	$12,3$	$5,3$

Con l'aggiunta della Tabella di Trasposizione, Iterative Deepening e Alphabeta migliorano in maniera differente. Anche in questo caso, la Tabella porta effettivi benefici a profondità maggiori o uguali a quattro.

Algoritmo	Miglioramento percentuale	Percentuale di hit
Alphabeta	15	22
Iterative Deepening	30	21

Iterative Deepening risulta essere molto più efficiente e superiore ad Alphabeta. Da notare che, sul numero totale di nodi esplorati, il $21\% - 22\%$ sono stati valutati recuperando il valore dalla Tabella di Trasposizione: questo dato mostra che, nel gioco degli Scacchi, il fenomeno della trasposizione non è affatto trascurabile.

5.8.3 Confronto fra le funzioni di valutazione

Prima di svolgere i test, i risultati attesi erano i seguenti: l'utilizzo della Lazy Evaluation avrebbe portato a ricerche più veloci, ma anche più imprecise; invece, l'utilizzo della funzione di valutazione a tabelle avrebbe portato a ricerche più lente, ma più accurate. Tali aspettative sono state smentite dai risultati ottenuti.

In media, il tempo di valutazione impiegato dalla Lazy Evaluation è 10 volte inferiore a quello impiegato dalla funzione di valutazione a tabelle. Nonostante ciò, in media, l'utilizzo della Lazy Evaluation non ha portato alcun incremento delle prestazioni, a fronte di mosse migliori più deboli. Anzi, vi sono stati casi in cui è avvenuto un notevole peggioramento. Infatti, è emerso che l'utilizzo della Lazy Evaluation porta il motore a generare molti più nodi e vengono prodotti meno tagli: la valutazione è troppo *piatta*. Come esposto nel capitolo dedicato alla Valutazione, viene valutato solo il materiale. Quindi, posizioni con materiale uguale, ma vantaggi posizionali differenti ricevono lo stesso valore. Inoltre, la distribuzione dei valori generati è concentrata in un intervallo ridotto e tali valori differisco di poco tra loro. Questo fenomeno inibisce l'efficacia della Finestra di Aspirazione sulla quale Alphabeta e Iterative Deepening si basano.

Invece, con l'utilizzo della funzione di valutazione a tabelle, si nota l'effetto contrario: l'analisi posizionale, anche se approssimativa, permette di

distinguere in maniera sufficientemente efficace posizioni aventi stesso materiale, ma vantaggi posizionali differenti. Quindi, la distribuzione dei valori generati è meno densa e l'intervallo dei valori possibili è più ampio, con la conseguenza che la Finestra di Aspirazione si restringe velocemente, permettendo di effettuare molti più tagli.

Bibliografia

[1] Cameron Browne. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligende and AI in Games*, 4(1), 2012.

[2] ChessProgrammingWiki.org. Perft Results. `https://www.chessprogramming.org/Perft_Results#cite_note-1`, 2020.

[3] Free Software Foundation. Xboard, 2007.

[4] R. M. Hyatt. Rotated bitmaps, a new twist on an old idea. *J. Int. Comput. Games Assoc.*, 22:213–222, 1999.

[5] Robert Hyatt. Crafty Chess Engine. `https://craftychess.com`.

[6] Stéphane Nicolet Joost VandeVondele. Stockfish Chess Engine. `https://github.com/official-stockfish`.

[7] Stefan-Meyer Kahlen. Universal chess interface specification(uci). `http://wbec-ridderkerk.nl/html/UCIProtocol.html`, 2004.

[8] Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293 – 326, 1975.

[9] D. Kopec and I. Bratko. The bratko-kopec experiment: A comparison of human and computer performance in chess. In M.R.B. Clarke, editor,

Advances in Computer Chess, Pergamon Chess Series, pages 57 – 72. Pergamon, 1982.

[10] Richard E. Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1):97 – 109, 1985.

[11] E. W. Kozdrowicki and D. W. Cooper. Coko iii: the cooper-koz chess program. *Commun. ACM*, 16:411–427, 1973.

[12] L. Lister and J. Schaeffer. An analysis of the conspiracy numbers algorithm. *Computers & Mathematics with Applications*, 27(1), 1994.

[13] T. A. Marsland and Fred Popowich. Parallel game-tree search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 7(4):442–452, April 1985.

[14] David Allen McAllester. Conspiracy numbers for min-max search. *Artif. Intell.*, 35(3):287–310, July 1988.

[15] J. von Neumann. Zur theorie der gesellschaftsspiele. *Mathematische Annalen*, 100:295–320, 1928.

[16] Jonathan Schaeffer. *Experiments in Search and Knowledge.* PhD thesis, CAN, 1986.

[17] Claude E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 1950.

[18] A. Tozzi. *Progetto e Realizzazione di un Programma Distribuito per il Gioco degli Scacchi.* PhD thesis, Università di Pisa, 1993.

[19] Albert Zobrist. A new hashing method with application for game playing. Technical report, Computer Science Department, The University of Wisconsin, Madison, WI, USA, 1970.